

MACHINE LEARNING IN FINANCE

MODULE 6 | LESSON 1

INTRODUCTION TO DEEP LEARNING

Reading Time 70 minutes

Prior Knowledge Machine learning

Keywords Deep learning, activation function, forward propagation, backpropagation

In this lesson, we are going to give a theoretical background of neural networks that will give you the intuition needed to master deep learning beyond the module. Once you are done with this lesson, you should be able to understand where deep learning is applicable, why it is needed, and the building blocks of neural networks. To ensure the accuracy provided by the neural networks is high, the concept of backpropagation will be introduced. The lesson will set the momentum for upcoming lessons that will heavily rely on this lesson.

1. Deep Learning

Deep learning is a field of machine learning that has been inspired by the functionality of the human brain neural networks (Zhang et al). In recent years, deep learning has attracted immense use cases due to its strong generalization capabilities and robust training power. When the data is huge, it has been shown that deep learning performs much better than classical machine learning algorithms.

Deep learning has successfully been applied in both classification and regression problems in computer visions, natural language processing, audio-visual recognition, and image processing. In the recent past, deep learning applications have been researched and implemented in the fields of finance, medicine, operations research, and banking. The applications of deep learning in finance have been fueled by the existence of vast financial data, which happens to be incomplete, uncertain, and inconsistent, thereby posing a huge challenge to the traditional statistical methods that were previously employed to perform financial data analysis.

Applications of deep learning in the financial world include:

- Credit risk modeling (Addo et al. 38; Modarres et al.)
- Forecasting of option prices, future prices, and share prices (Sezer et al.; Yao et al. 455-466)
- Predicting bankruptcy (Mai et al. 743-758; Qu et al. 895-899)

The advancement of deep learning and machine learning in the areas of finance has largely been facilitated by:

- Evolution of computing power.
- Advancement of mathematical theory by machine learning researchers.
- Accessibility of technology.
- Breakthrough achieved in other domains of deep learning such as in games, natural language processing, image processing, etc.

In financial engineering, the following problems can be solved using deep learning algorithms:

- Deep hedging (Buehler et al. 1271-1291)
- Deep calibration (Nixon et al.)
- Deep simulation. (Maeda et al. 71)

Now that we have introduced deep learning and its application areas, in this lesson, we will learn about the following topics:

1. Deep learning
2. Forward propagation
3. Backward propagation

Deep learning derives its name from artificial neural networks (ANN) having many layers where each layer is designed to learn the intricate patterns found in the data. As pointed out earlier, it is nowadays common to build neural networks with thousands of layers, which has been made possible by the advancement in computational power. Because of this fact where ANN tend to use deep layers to learn, the name deep learning was adopted for such algorithms (Zhang et al).

We started by saying that deep learning is a branch of machine learning. The question we might then ask ourselves is, what differentiates the two?

Whilst the performance of classical machine learning algorithms is highly dependent on the correct choice of features and feature engineering techniques applied to the dataset, a task that is no mean feat, in deep learning, this is well taken care of by the layers that have the capability to learn complex relationships of the data. When our dataset is small, the deep learning algorithms are not preferred as they tend to overfit on the dataset and thus provide bad generalization. In such cases, machine learning performs much better than deep learning models.

Having discussed the definition of deep learning and where it is used, let us now learn the building blocks of deep learning algorithms.

1.1 Neurons

A neuron forms the fundamental computational unit of artificial neural networks. Below we examine the functionality of artificial neural networks, let us assume that the predictor variables are x_1 , x_2 , x_3 and x_4 and we are predicting y .

Since the variables will usually not be equally important in predicting the output, we assign weight multipliers to all the inputs to signify importance. Therefore, summing the inputs that have been multiplied by the weights will yield $x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + x_4 \times w_4$. By adding a bias term to the above equation we get $z = (x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + x_4 \times w_4) + b$

The above equation resembles that of a linear regression. The difference between a neuron and a linear regression is the introduction of a non-linear function called an activation function that is applied to z to predict the output, that is, $y = f(z)$

```
In [ ]: import matplotlib.pyplot as plt
        from nnv import NNV

        print("\033[1m" + "Fig 1: A Neural Network" + "\033[0m")
        layersList = [
            {"title": "inputs \n + bias", "units": 5, "color": "darkBlue"},
            {"title": "$z$", "units": 1},
            {"title": "$f(z)$", "units": 1, "edges_color": "red", "edges_width": 2},
            {"title": "output", "units": 1, "color": "darkBlue"},
        ]

        NNV(layersList).render(save_to_file="my_example1.png")
        plt.show()
```

So, in summary, given inputs X , a neuron will pair them with weights W and a bias term b , which after adding them will be acted upon by an activation function to yield the output y .

1.2 Neural Network Layers

Nearly all tasks we will face in the industry will need us to stack more layers to learn the intricate relationship present in the data. The organization of layers in a neural network is such that it can allow information to flow from one layer to the next. The layers in a simple neural network are:

- Input layer
- Hidden layer
- Output layer

This can be shown in the diagram below:

```
In [ ]: print("\033[1m" + "Fig 2: Two Layer Neural Network" + "\033[0m")
        layersList = [
            {"title": "input layer", "units": 3, "color": "darkBlue"},
            {"title": "hidden layer", "units": 4},
            {"title": "output layer", "units": 1, "color": "darkBlue"},
        ]

        NNV(layersList).render(save_to_file="my_example2.png")
        plt.show()
```

The input layer consists of the observed variables that are passed to the model. They can be raw and unprocessed data straight from the data source or featured engineered datasets.

The hidden layer is the layer that processes data passed from the input layer to derive complex patterns present in the data. There can be thousands of hidden layers present in a neural network. The processed data is then passed to the output layer.

The output layer gives the output of the modeling exercise. There can be more than one neuron in the output layer depending on the problem being solved.

1.3 Activation Functions

As pointed out earlier, activation functions are used to introduce non-linearity to the input so as to learn the underlying relationship in the data, thus allowing for complex fitting. Without a non-linear transformation, the neural network is a linear regression model.

We will now explore the most commonly used activation functions.

1.3.1 Sigmoid Function (Logistic Function)

The sigmoid function is defined as $f(x) = \frac{1}{1+e^{-x}}$ It gives an S-shaped curve as below:

```
In [ ]: import math

import matplotlib.pyplot as plt
import numpy as np

def sigmoid(x):
    a = []
    for item in x:
        a.append(1 / (1 + math.exp(-item)))
    return a

x = np.arange(-5.0, 5.0, 0.2)
sig = sigmoid(x)

print("\033[1m" + "Fig 3: A sigmoid function curve" + "\033[0m")
plt.plot(x, sig)
# `plt.suptitle`("Fig. 3: A sigmoid function curve")
plt.show()
```

The sigmoid function is differentiable and monotonic and it outputs values between 0 and 1. We can write the sigmoid function in Python as the following:

```
In [ ]: import numpy as np
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

1.3.2 Tanh Function

The tanh function expressed as $f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$ resembles the sigmoid function above with the difference being that it outputs values between -1 and 1 as shown below.

```
In [ ]: import matplotlib.pyplot as plt

def tanh(x):
    a = []
    for item in x:
        a.append((1 - math.exp(-2 * item)) / (1 + math.exp(-2 * item)))
    return a

x = np.arange(-5.0, 5.0, 0.2)
tanva = tanh(x)

print("\033[1m" + "Fig 4: A tanh function curve" + "\033[0m")
plt.plot(x, tanva)
plt.show()
```

It can be implemented in Python as shown below:

```
In [ ]: def tanh(x):
    return (1 - np.exp(-2 * x)) / (1 + np.exp(-2 * x))
```

1.3.3 Rectified Linear Unit (ReLU) Function

The ReLU function is mathematically expressed as $f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ or $f(x) = \max\{0, x\}$ whose outputs are non-negative numbers.

It can pictorially be visualized below.

```
In [ ]: import numpy as np

x = np.linspace(-10, 10, 1000)
y = np.maximum(0, x)

print("\033[1m" + "Fig 5: A ReLU function curve" + "\033[0m")
plt.figure(figsize=(10, 5))
plt.plot(x, y)
plt.legend(["Relu"])
# `plt.suptitle`("Fig. 5: A ReLU function curve")
plt.show()
```

The Python code for ReLU is:

```
In [ ]: def ReLU(x):
    if x < 0:
```

```

        return 0
    else:
        return x

```

1.3.4 The Leakey ReLU Function

The Leakey ReLU is a modification of the ReLU function intended to tackle the dying ReLU problem. It is mathematically expressed as: $f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ where α usually takes the value 0.01.

It is implemented in Python as:

```

In [ ]: def LeakeyReLU(x, alpha=0.01):
        if x < 0:
            return alpha * x
        else:
            return x

```

Diagrammatically, we can visualize it as:

```

In [ ]: import matplotlib.pyplot as plt
        import numpy as np

        # Leaky Rectified Linear Unit (Leaky ReLU) Activation Function
        def leaky_ReLU(x):
            data = [max(0.05 * value, value) for value in x]
            return np.array(data, dtype=float)

        # Generating data For Graph
        x_data = np.linspace(-10, 10, 100)
        y_data = leaky_ReLU(x_data)

        # Graph
        print("\033[1m" + "Fig 6: A Leakey ReLU function curve" + "\033[0m")
        plt.plot(x_data, y_data)
        plt.legend(["leaky_ReLU"])
        plt.grid()
        plt.show()

```

1.3.5 The Softmax Function

The softmax function generalizes the sigmoid function, and it is used in multi-class classification problems while logistic function is suitable for binary classes. It is expressed as: $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ The Python code to produce the above equation is:

```

In [ ]: def softmax(x):
        return np.exp(x) / np.exp(x).sum(axis=0)

```

Other commonly used activation functions are:

a) The exponential linear unit function given by $f(x) = \begin{cases} \alpha (e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ b) The swish function given by $f(x) = x \sigma(x)$ where $\sigma(x)$ is the sigmoid function.

Now that we have learned the founding blocks of deep learning, let us now apply them in the next section.

2. Forward Propagation

In the previous section, we said that neural networks learn by stacking neurons in layers. A neural network is said to have n -layers if the number of layers excluding the input layer are n . Therefore, a two-layer neural network will have one input layer X , one hidden layer h , and one output layer y as shown below:

```
In [ ]: print("\033[1m" + "Fig 7: A Two Layer Neural Network" + "\033[0m")
layersList = [
    {"title": "input layer", "units": 3, "color": "darkBlue"},
    {"title": "hidden layer", "units": 4},
    {"title": "output layer", "units": 1, "color": "darkBlue"},
]

NNV(layersList).render(save_to_file="my_example3.png")
plt.show()
```

Take a case where our input variables are three, that is, x_1 , x_2 and x_3 as in the diagram above. The number of neurons in the input layer will therefore be three, and in this example, we will also assume the number of neurons in the hidden layer and output layer to be four and one respectively. As earlier indicated, the input variables will be multiplied by their corresponding weights, then we add a bias term before passing this sum to the hidden layer where the activation function will be used.

The obvious question we might be asking ourselves is given the inputs, where do we get the corresponding weights? Practically, it will be impossible to know which of the inputs will be more important in prediction and by what proportion. We therefore assign weights and the bias term using random numbers that we will denote as W_{xh} and b_h respectively, where the dimensions of the weight matrix will have a number of columns equal to the number of neurons in the input layer and a number of rows equal to the number of neurons in the hidden layer so as to ensure matrix multiplication compatibility. Thus, in the presented example, the dimension of W_{xh} is 3×4 and therefore $z_1 = X W_{xh} + b_h$. We pass this equation to the hidden layer where it is acted upon by the activation function. If the activation function in this case is the sigmoid function, then we have $a_1 = \sigma(z_1)$. The outputs a_1 is again multiplied by a new weight matrix and added to a new bias term that is now passed to the output layer. By denoting the weight matrix and bias term here by W_{hy} and b_y respectively, the dimensions of W_{hy} will be 4×1 since the hidden layer has 4 neurons and the output layer has one neuron.

The result is given by $z_2 = a_1 W_{hy} + b_y$

Finally, we apply another activation function to z_2 to get the output value $\hat{y} = \sigma(z_2)$. The above process is what we call the forward propagation.

In summary, the mathematical steps involved in a two-layer neural network is as follows:

$$\begin{cases} z_1 = X W_{xh} + b_h \\ a_1 = \sigma(z_1) \\ z_2 = a_1 W_{hy} + b_y \\ \hat{y} = \sigma(z_2) \end{cases}$$

A Python function implementation of the above steps is as shown below:

```
In [ ]: def forward_prop(X, b_h, b_y, W_x_h, W_h_y):  
        z_1 = np.dot(X, W_x_h) + b_h  
        a_1 = sigmoid(z_1)  
        z_2 = np.dot(a_1, W_h_y) + b_y  
        y_hat = sigmoid(z_2)  
  
        return y_hat
```

Are we done? Apparently not. We have to compare our approximated output with the actual output. How do we therefore know that our model is performing up to expectation? We define a cost/loss function. Just as there are many activation functions, there exist many cost functions. To build the intuition in the next section, we will use the cost function below $J = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$ the sum of the squared difference between the actual output and the predicted output where n is the number of variables used in training.

Now that we have learned how to predict the output of a neural network, in the next section, we are going to fine-tune this to get better approximations.

3. Backpropagation

At this time, the question that is lingering in our heads might be how does a neural network learn to predict the correct output? Where is the cost function coming in in all of this? A short answer to this is that we aim to have the cost function as small as possible, and therefore for any iteration we will encounter, we will seek to minimize the cost function. But why? If the cost function is large, then it means that the difference between the predicted and actual outputs is high.

The only way we can minimize the cost function is to change some parameters, but we cannot change the input and output values. Therefore, we can only change the weight and bias terms since they were initially assigned random numbers. The role of backpropagation will be to help the neural network learn how to update the weights in such a manner so as to minimize the cost function thereby increasing the accuracy of our prediction. We use a method called gradient descent to update the weights.

Gradient descent is an optimization algorithm commonly used in machine learning to find the optimal values of weights that will minimize the cost function or errors.

To understand how gradient descent finds the optimal weights, let us provide an intuitive explanation. From its name "Gradient descent," think of going down a slope. The

technique tries to find a combination of parameters in a curve that will ensure the output gives a low-cost value.

Since our interest is to find the optimal weight, consider the curve below as a function of w , that is, $E(w)$ as shown below:

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np

w = np.linspace(-3, 3, num=100)

Ew = []
for i in range(len(w)):
    Ew.append(w[i]**4 - 5 * w[i]**2 - 3 * w[i])

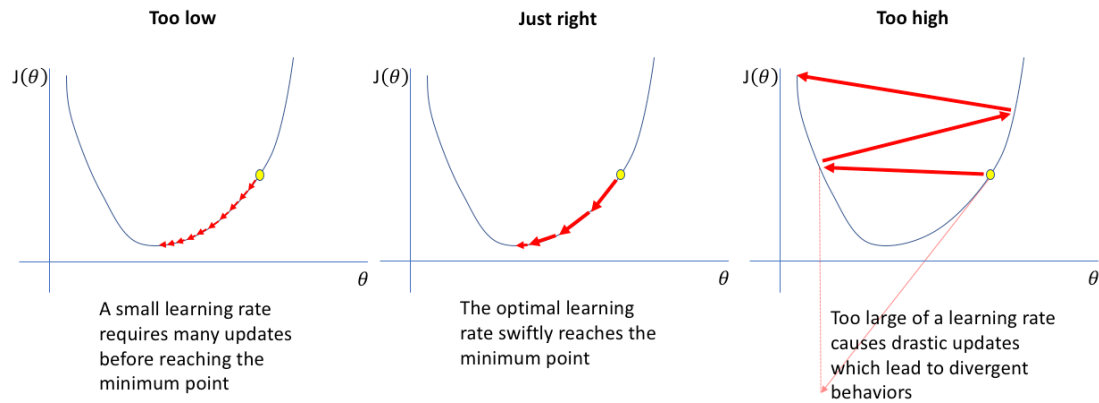
print("\033[1m" + "Fig 8: A Non-convex Curve" + "\033[0m")
plt.plot(w, Ew)
plt.grid()
plt.axvline()
plt.axhline()
plt.show()
```

Our ideal will be to hit the global minimum, but gradient (the lowest trough in the curve) descent will not necessarily get to this point, though it will get close. We start first by selecting initial values of w . The gradient descent will attempt to reach the local minimum point, and since it will only descend and not climb, by increasing or decreasing the value of w gradually in small steps, we will see it go down the slope toward the global minimum. We refer to these steps as the learning rate denoted by α .

By repeating this process over many iterations, the gradient descent will give us optimal weights that ensure the cost is at a minimum. Remember that by forward propagation we stopped at the output layer. By back-propagation, we will move in the reverse order, that is, from the output layer to the input layer while calculating the cost function's gradient that minimizes the error. Having calculated the gradients, we can now proceed to update the weights as follows: $W_{\text{new}} = W - \alpha \frac{\partial J}{\partial W}$ where α is the learning rate.

The learning rate guides the type of step to be taken, that is, a small learning rate infers we take a small step down the slope and therefore the gradient descent process becomes slow. The inverse is also true as shown in the diagrams below.

Fig. 9: Gradient Descent with Different Learning Rates



Source: [Jeremy Jordan, Setting the Learning Rate of Your Neural Network, 2018](#)

This whole procedure of moving from the output layer to the input layer while updating the weights using an optimization algorithm is called back-propagation.

In the two-layer neural network example we considered in the last section, we had two weights, W_{hx} that feeds inputs to hidden layer and W_{hy} that feeds to the output layer from the hidden layer. To find the optimal weights that gives us minimal errors, we will find the derivative of J with respect to the weights.

We will start with W_{hy} since we are moving backwards, from the output layer towards the input layer. Recall that cost function J is given by: $J = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$ and $\hat{y} = \sigma(z_2)$ where $z_2 = a_1 W_{\text{hy}} + b_y$. To find the derivative of J with respect to W_{hy} we will use the chain rule, that is,

$$\frac{\partial J}{\partial W_{\text{hy}}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_2} \times \frac{\partial z_2}{\partial W_{\text{hy}}} \quad (1)$$

Note that: $\frac{\partial J}{\partial \hat{y}} = (y - \hat{y})$ and $\frac{\partial \hat{y}}{\partial z_2} = \sigma'(z_2)$ where σ' denotes the derivative of the sigmoid activation function.

The derivative of the sigmoid function $\sigma(z) = \frac{1}{1 + e^{-z}}$ is $\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$. Finally, $\frac{\partial z_2}{\partial W_{\text{hy}}} = a_1$. Combining the derivatives back to the equation (1) above, we get

$$\frac{\partial J}{\partial W_{\text{hy}}} = (y - \hat{y}) \times \sigma'(z_2) \times a_1 \quad (2)$$

Next, we now find the derivative of J with respect to the weight W_{hx} . Recall that: $\hat{y} = \sigma(z_2)$, $z_2 = a_1 W_{\text{hy}} + b_y$, $a_1 = \sigma(z_1)$, $z_1 = X W_{\text{hx}} + b_x$. By the chain rule, we get

$$\frac{\partial J}{\partial W_{\text{hx}}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial W_{\text{hx}}} \quad (3)$$

where: $\frac{\partial z_2}{\partial a_1} = W_{\text{hy}}$ and $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$. So

$$\frac{\partial J}{\partial W_{\text{hx}}} = (y - \hat{y}) \times \sigma'(z_2) \times W_{\text{hy}} \times \sigma'(z_1) \times X \quad (4)$$

Therefore: $\frac{\partial J}{\partial W_{\text{hx}}} = (y - \hat{y}) \times \sigma'(z_2) \times W_{\text{hy}} \times \sigma'(z_1) \times X$

Since we now have the gradients, we can now update the weights as: $W_{hy} = W_{hy} - \alpha \frac{\partial J}{\partial W_{hy}}$ (5) $W_{xh} = W_{xh} - \alpha \frac{\partial J}{\partial W_{xh}}$ (6)

Let us now implement backpropagation in Python. We will define dW_{hy} and dW_{xh} to represent $\frac{\partial J}{\partial W_{hy}}$ and $\frac{\partial J}{\partial W_{xh}}$ respectively.

We first write a function for the derivative of the sigmoid function.

```
In [ ]: def sigmoid_derivative(z):
        return np.exp(-z) / ((1 + np.exp(-z)) ** 2)
```

Define $(y - \hat{y}) \sigma'(z_2)$ as:

```
delta2 = np.multiply((y_hat - y), sigmoid_derivative(z_2))
```

The gradient with respect to W_{hy} is therefore:

```
dWhy = np.dot(a1.T, delta2)
```

and with respect to W_{xh} is:

```
delta1 = np.dot(delta2, Why.T) * sigmoid_derivative(z1)
```

```
dWxh = np.dot(X.T, delta1)
```

We therefore update the weights accordingly:

```
Wxh = Wxh - alpha * dWxh
```

```
Why = Why - alpha * dWhy
```

So, the function becomes:

```
In [ ]: def backward_prop(y_hat, z_1, a_1, z_2, alpha, W_x_h, W_h_y, X):
        delta2 = np.multiply((y_hat - y), sigmoid_derivative(z_2))
        d_W_h_y = np.dot(a_1.T, delta2)

        delta1 = np.dot(delta2, W_h_y.T) * sigmoid_derivative(z_1)
        d_W_x_h = np.dot(X.T, delta1)

        W_x_h = W_x_h - alpha * d_W_x_h
        W_h_y = W_h_y - alpha * d_W_h_y

        return W_x_h, W_h_y
```

4. Conclusion

In this lesson, we started by introducing the concept of deep learning and its application areas in financial engineering. We started by discussing the fundamental building blocks of deep learning, after which we discussed how neural networks get the output

predictions using forward prediction. Then, we fine-tuned the algorithm by introducing the concept of gradient descent in back-propagation.

In the next lesson, we will put these concepts together as we work on examples and compare the performance of deep learning and other algorithms.

References

1. Addo, Martey Peter et al. "Credit Risk Analysis Using Machine and Deep Learning Models." *Risks*, vol. 6, no. 2, 2018, pp. 1-20.
2. Buehler, Hans et al. "Deep Hedging." *Quantitative Finance*, vol. 19, no. 8. 2019, pp. 1271-1291.
3. Maeda, Iwao et al. "Deep Reinforcement Learning in Agent Based Financial Market Simulation." *Journal of Risk and Financial Management*, vol. 13, no. 4, 2020.
4. Mai, Feng, et al. "Deep Learning Models for Bankruptcy Prediction Using Textual Disclosures." *European Journal of Operational Research*, vol. 274, no. 2, 2019, pp. 743-758.
5. Modarres, Ceena et al. "Towards Explainable Deep Learning for Credit Lending: A Case Study." *arXiv preprint*, arXiv:1811.06471, 2018.
6. Nixon, Jeremy et al. "Measuring Calibration in Deep Learning." *CVPR Workshops*, vol. 2, no. 7, 2019.
7. Sezer, Omer Berat et al. "Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review: 2005-2019." *Applied Soft Computing*, vol. 90, 2020, 1-63.
8. Yao, Jingtao et al. "Option Price Forecasting Using Neural Networks." *Omega*, vol. 28, no. 4, 2000, pp. 455-466.
9. Qu, Yi et al. "Review of Bankruptcy Prediction Using Machine Learning and Deep Learning Techniques." *Procedia Computer Science*, vol. 162, 2019, pp. 895-899.
10. Zhang, Aston et al. "Dive into Deep Learning." *arXiv preprint* arXiv:2106.11342, 2021.
<https://d2l.ai/d2l-en.pdf>

MACHINE LEARNING IN FINANCE

MODULE 6 | LESSON 2

APPLICATION OF DEEP LEARNING IN PREDICTING YIELD CURVES

Reading Time	100 minutes
Prior Knowledge	Machine learning, Introduction to deep learning
Keywords	Yield curve, Logistic Regression, Multi-Linear Perceptron

In the previous lesson, we introduced the concept of deep learning and its applications. In this lesson, we will build on deep learning concepts by solving a real-world problem of forecasting yield curves using deep learning algorithms and comparing our results to a high-order polynomial regression.

1. Recap of Yield Curves

We can define yield curves as a graph showing interest rates (bonds) having different maturity rates. Yield curves are important in the financial world for these reasons:

1. They are used as a benchmark for setting mortgage rates, and;
2. They provide a good indicator for predicting economic growth.

There are three types of yield curves:

- Normal yield curve
- Inverted yield curve
- Flat yield curve

We will review each one of them below.

1.1 Normal Yield Curve

The graph of a normal yield curve shows an upward trajectory that implies longer maturity date bonds will pay more in interest (have higher yield) as compared to shorter maturity date bonds.

A normal yield curve can be interpreted as people having confidence that the economy will perform well in the future, and we would therefore expect the central banks to increase the interest rate in that future date. The central banks raise interest rates in response to the inflationary pressures caused by an expanding economy.

1.2 Inverted Yield Curve

An inverted yield curve is a downward sloping curve that implies a recession is about to happen. An inverted yield curve means that people believe interest rates will be lower than they currently are, and we therefore expect the central banks to cut interest rates so as to encourage borrowing of more money by investors, consumers, and businesses. This in turn leads to economic growth.

1.3 Flat Yield Curve

We get a flat yield curve when an economy is shifting from one state to another. For example, when an expanding economy starts contracting we see that a yield curve that was initially normal begins to flatten due to people expecting an economic downturn and thus a drop in long-term interest rates. The opposite is also true.

Now that we have a grasp of how useful yield curves are, in the subsequent sections, we are going to forecast yield curves using conventional machine learning algorithms and a deep learning algorithm.

2. Forecasting Yield Curves

This case study is a time series problem whose foundation we studied in the Econometrics course. It is also a supervised machine learning problem since we have outputs that our model will be learning from. The predicted variables in this case will be multiple outputs.

To solve this problem, we will follow an industry standard methodology: Cross-Industry Process for Data Mining (CRISP-DM) used for solving data science problems.

With the exception of the deployment stage, which is left for curious students to practice on their own, the other steps will be demonstrated here.

2.1 Problem Understanding

The focus of this study is to find and understand factors that affect movements of the yield curve. Among the variables we will use are the historical data of the yield curve, federal debt percentage that is held by investors, and the corporate spread. The dependent variable will be the 1-month, 5-year and 30-year tenure of the yield curve.

We sourced our data from the Yahoo and FRED websites. In this problem, we will use polynomials of various degrees to forecast the yield curves.

2.2 Data Understanding

We start by loading packages that we will use throughout this lesson as shown below.

```
In [ ]: # Load Libraries
# Global Libraries
# Disable the warnings
import warnings

# Time series Models
from datetime import datetime

import numpy as np
import pandas as pd
import pandas_datareader.data as web

# Plotting
import seaborn as sns

# Libraries for Statistical Models
import statsmodels.api as sm
from matplotlib import pyplot
from pandas.plotting import scatter_matrix

# Regression ML
from sklearn.linear_model import LinearRegression

# Loss Function
from sklearn.metrics import mean_squared_error

# Modeling
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    cross_val_score,
    train_test_split,
)
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import PolynomialFeatures

warnings.filterwarnings("ignore")
```

Next, we load the data using the pandas `datareader` function:

```
In [ ]: # downloading the data
start = datetime(2007, 6, 30)
end = datetime(2022, 6, 30)

data = [
    "DGS1M0",
    "DGS3M0",
    "DGS1",
    "DGS2",
    "DGS5",
    "DGS7",
    "DGS10",
    "DGS30",
    "TREAST", # -- U.S. Treasury securities held by the Federal Reserve ( Milli
    "FYGFDPU", # -- Federal Debt Held by the Public ( Millions of Dollars )
    "FDHBFIN", # -- Federal Debt Held by Foreign and International Investors (
    "GFDEBTN", # -- Federal Debt: Total Public Debt ( Millions of Dollars )
    "BAA10Y", # -- Baa Corporate Bond Yield Relative to Yield on 10-Year
]
```

```

data = web.DataReader(data, "fred", start, end).dropna(how="all").ffill()
data["FDHBFIN"] = data["FDHBFIN"] * 1000
data["GOV_PCT"] = data["TREAST"] / data["GFDEBTN"]
data["HOM_PCT"] = data["FYGFDPUN"] / data["GFDEBTN"]
data["FOR_PCT"] = data["FDHBFIN"] / data["GFDEBTN"]

```

We then define independent and dependent variables as mentioned in the previous subsection, and we also make an assumption that trading only happens on weekdays. As such, we will calculate the lagged version of the variables using a 5 trading day lag.

```

In [ ]: data.rename(
    columns={
        "DGS1M0": "1m",
        "DGS3M0": "3m",
        "DGS1": "1y",
        "DGS2": "2y",
        "DGS5": "5y",
        "DGS7": "7y",
        "DGS10": "10y",
        "DGS30": "30y",
    },
    inplace=True,
)
return_period = 5

Y = data.loc[:, ["1m", "5y", "30y"]].shift(-return_period)
Y.columns = [col + "_pred" for col in Y.columns]

X = data.loc[
    :,
    [
        "1m",
        "3m",
        "1y",
        "2y",
        "5y",
        "7y",
        "10y",
        "30y",
        "GOV_PCT",
        "HOM_PCT",
        "FOR_PCT",
        "BAA10Y",
    ],
]

df = pd.concat([Y, X], axis=1).dropna().iloc[:, :return_period, :]
Y = df.loc[:, Y.columns]
X = df.loc[:, X.columns]

df.head(10)

```

2.3 Exploratory Data Analysis (EDA)

EDA allows us to understand the hidden structure of our dataset by visualizing trends, relationships, and patterns that would have escaped the human eye. With EDA, we can already start to draw conclusions from our data. Exploring data constitutes a crucial step

in our data analysis. In this subsection, we will show graphical exploratory data analysis that involves taking data from the dataframe and representing it graphically. Students are required to study the graphs closely to understand what they mean.

```
In [ ]: df.shape
```

This data has 243 observations and 15 variables.

```
In [ ]: df.describe()
```

We now plot the predicted variables to observe their behavior.

```
In [ ]: # print('\033[1m' + 'Fig 5: Yield Curve Prediction' + '\033[0m' )
Y.plot(style=["-", "--", "*"])
pyplot.suptitle(
    "Fig. 5: Yield Curve Prediction",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Notice that in the beginning of 2017, deviations for the 1-month, 5-year, and 30-year rates were high but steadily reduced afterwards. As of 2022, the medium- and long-term rates, that is 5 years and 30 years, were approximately equal but very much deviated from the short-term rate. Therefore, these time series trends show non-stationary behavior.

```
In [ ]: # Scatterplot Matrix
pyplot.figure(figsize=(15, 15))
scatter_matrix(df, figsize=(15, 16))
pyplot.suptitle(
    "Fig. 6: Scatter Plot Matrix",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

From the above scatter plots, it is evident that there exists a strong linear relationship between the predicted variables and other tenures in the dataset.

```
In [ ]: # correlation
correlation = df.corr()
pyplot.figure(figsize=(16, 16))
pyplot.title("Correlation Matrix")
pyplot.suptitle(
    "Fig. 7: Correlation Plot",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="cubehelix")
pyplot.show()
```

The correlation plot above can be interpreted in a similar manner as the scatter plot figure discussed earlier.

We now do an EDA on the predicted variables as a time series plot.

First is the 1-month time series analysis.

```
In [ ]: temp_Y = df["1m_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 8: Time Series Trend Analysis for 1 month period",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Then, the 5-year analysis:

```
In [ ]: temp_Y = df["5y_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 9: Time Series Trend Analysis of the 5-Year predicted variable",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Finally, the 30-year variable

```
In [ ]: temp_Y = df["30y_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 10: Time Series Trend Analysis of the 30-Year predicted variable",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

3. Data Preparation

The dataset in this exercise does not need much preprocessing, and therefore, we are going to use it as is. Note that data preparation involves transforming the data, for

example, by scaling so that we remove bias when running machine learning algorithms on our datasets.

4. Modeling

4.1 Splitting the Dataset into Train and Test Samples

We will split the data into 80% for training (modeling) the dataset and 20% to validate our model. To evaluate our algorithms, we choose the mean squared error since this is a regression exercise.

```
In [ ]: # split out validation dataset for model validation
test_size = 0.2

seed = 42
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=test_size, random_state=seed
)
```

We use cross-validation to validate the model while training on the limited data sample.

```
In [ ]: # test options for regression
num_folds = 10
scoring = "neg_mean_squared_error"
```

4.2 Forecasting Yield Curves: Fitting High Order Polynomials

Linear regression assumes that there exists a linear relationship between the dependent and independent variables. To account for non-linearity in our dataset, we choose to employ polynomial regression by adding polynomial terms: $y = w_1 x + w_2 x^2 + \dots + w_m x^m + b$ where m is the degree of our polynomial.

In our case study, we will compare the performance of 1-order, 2-order, and 3-order polynomial regressions. We use transformer `PolynomialFeatures` from scikit-learn to add quadratic terms.

```
In [ ]: lr = LinearRegression()
lr.fit(X_train, y_train)

pred = lr.predict(X_test)

MSE = mean_squared_error(y_test, pred)

print("linear regression mean squared error:", MSE)
```

```
In [ ]: # Quadratic
poly = PolynomialFeatures(degree=2)
X_train2 = poly.fit_transform(X_train)
X_test2 = poly.fit_transform(X_test)

model = lr.fit(X_train2, y_train)
```

```

print(
    "quadratic regression mean squared error:",
    mean_squared_error(y_test, model.predict(X_test2)),
)

# Cubic
poly = PolynomialFeatures(degree=3)
X_train3 = poly.fit_transform(X_train)
X_test3 = poly.fit_transform(X_test)

model = lr.fit(X_train3, y_train)
print(
    "degree 3 polynomial regression mean squared error:",
    mean_squared_error(y_test, model.predict(X_test3)),
)

```

Clearly, linear regression outperforms higher-order polynomials in the prediction task, and we would therefore use cross-validation with it to compare our output with a deep learning algorithm.

4.3 Forecasting Yield Curves: Cross-Validation

In k -fold cross-validation, data is split randomly into k folds and without replacement. We train our data into $k-1$ folds and validate the model on the remaining one fold also called a test fold. The method is repeated k -terms, meaning we will have k results. k -fold cross-validation is typically used for tuning our model.

We estimate the average performance E by $E = \frac{1}{k} \sum_{i=1}^k E_i$

Cross-validation can be applied to any model, and we will therefore first discuss neural networks in the next subsection and apply cross-validation in our neural network algorithm to fine-tune its performance.

4.4 Forecasting Yield Curves - Backpropagation

In the last lesson, we studied backpropagation in detail and we will put our skills to practice in this section. We will later apply an artificial neural network algorithm, (MLP), which is part of the machine learning library in `sklearn`.

Before we proceed, let us familiarize ourselves with some common terminology in deep learning:

- **Forward pass:** It is synonymous with forward propagation, that is, moving from the input layer to the output layer.
- **Backward pass:** It is synonymous with backward propagation, that is, moving from the output layer to the input layer.
- **Epoch:** It is the number of times our model trains the whole data, that is, one epoch is equivalent to one forward pass and one backward pass for the training data.
- **Iterations:** Number of times we perform forward pass and backward pass.
- **Batch size:** It is the subset of training data that we use in one forward pass and one backward pass.

In our model, we will use a two-layered neural network where the input layer has twelve variables, that is, the predictor variables. The hidden layer has 20 nodes for purposes of demonstration, and the output layer has three nodes corresponding to the predicted variables. So we start by defining the number of nodes in each layer.

```
In [ ]: from nnv import NNV

print("\033[1m" + "Fig 12: A Two Layer Neural Network" + "\033[0m")
layersList = [
    {"title": "inputs \n + bias", "units": 12, "color": "darkBlue"},
    {"title": "hidden layer", "units": 20},
    # {"title": "$f(z)$", "units": 1, "edges_color": "red", "edges_width": 2},
    {"title": "output", "units": 3, "color": "darkBlue"},
]

NNV(layersList).render(save_to_file="my_example.png")
pyplot.show()
```

```
In [ ]: num_input = 12
num_hidden = 20
num_output = 3
```

Next, we initialize values of weights and bias randomly. We start by initializing the input to hidden layer weights.

```
In [ ]: W_x_h = np.random.randn(num_input, num_hidden)
b_h = np.zeros((1, num_hidden))
```

Then, we initialize the hidden layer to output layer weights and bias.

```
In [ ]: W_h_y = np.random.randn(num_hidden, num_output)
b_y = np.zeros((1, num_output))
```

We can now define our activation function and its derivative, in this case, the sigmoid function.

```
In [ ]: def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return np.exp(-z) / ((1 + np.exp(-z)) ** 2)
```

As in the previous section, we then write forward propagation and backward propagation functions.

```
In [ ]: def forward_prop(X, W_x_h, W_h_y):
    z_1 = np.dot(X, W_x_h) + b_h
    a_1 = sigmoid(z_1)
    z_2 = np.dot(a_1, W_h_y) + b_y
    y_hat = sigmoid(z_2)

    return z_1, a_1, z_2, y_hat
```

```
def back_prop(y_hat, z_1, a_1, z_2):
    delta2 = np.multiply(-(y_train - y_hat), sigmoid_derivative(z_2))
    dJ_d_W_h_y = np.dot(a_1.T, delta2)
    delta1 = np.dot(delta2, W_h_y.T) * sigmoid_derivative(z_1)
    dJ_d_W_x_h = np.dot(X_train.T, delta1)

    return dJ_d_W_x_h, dJ_d_W_h_y
```

The cost function we use is the squared error as shown below:

```
In [ ]: def cost_function(y, y_pred):
        return 0.5 * np.sum((y - y_pred) ** 2)
```

Before training our model, we set the learning rate and number of iterations.

```
In [ ]: alpha = 0.01
        num_iterations = 10000
```

We now train the neural network.

```
In [ ]: cost = []

        for i in range(num_iterations):
            z_1, a_1, z_2, y_hat = forward_prop(X_train, W_x_h, W_h_y)
            dJ_d_W_x_h, dJ_d_W_h_y = back_prop(y_hat, z_1, a_1, z_2)

            # update weights
            W_x_h = W_x_h - alpha * dJ_d_W_x_h
            W_h_y = W_h_y - alpha * dJ_d_W_h_y

            # compute cost
            c = cost_function(y_train, y_hat)

            cost.append(c)
```

The plot of the cost function is presented below:

```
In [ ]: pyplot.grid()
        pyplot.plot(range(num_iterations), cost)

        pyplot.title("Cost Function")
        pyplot.xlabel("Training Iterations")
        pyplot.ylabel("Cost")
        pyplot.suptitle(
            "Fig. 13: Cost Function Plot",
            fontweight="bold",
            horizontalalignment="right",
            fontsize=15,
        )
        pyplot.show()
```

As compared to the regression models in the first subsection of this section, the neural network model performs poorly. We can always improve this performance by increasing the number of nodes in the hidden layer or increasing the number of hidden layers to allow for complex modeling of the data.

Now that we have a clear understanding of how neural networks work, let's implement a neural network algorithm called **Multilayer Perceptron** to predict yield curves.

We will use Multilayer Perceptron (MLP) as an example of a deep neural network that is fully connected.

As discussed above, an MLP algorithm follows these steps:

1. Given an input layer, training data is forward-propagated through a deep neural network to get an output.
2. Then, calculate the loss to check whether it is at a minimum.
3. The loss is backpropagated and the model updated.

Finally, once we are satisfied with the output of the above three steps, we use forward propagation to get our final output.

Below is a demonstration of the use of cross-validation and backpropagation in forecasting yield curves.

```
In [ ]: # test options for regression
num_folds = 10
scoring = "neg_mean_squared_error"
```

```
In [ ]: # spot check the algorithms
models = []

# Linear Regression Algorithm
models.append(("LR", LinearRegression()))

# Deep Learning Algorithm
models.append(("MLP", MLPRegressor()))
```

```
In [ ]: kfold_results = []
names = []
validation_results = []
train_results = []
for name, model in models:
    kfold = KFold(n_splits=num_folds)
    # converted mean square error to positive. The lower the better
    cv_results = -1 * cross_val_score(
        model, X_train, y_train, cv=kfold, scoring=scoring
    )
    kfold_results.append(cv_results)
    names.append(name)

    # Finally we Train on the full period and test against validation
    res = model.fit(X_train, y_train)
    validation_result = np.mean(np.square(res.predict(X_test) - y_test))
    validation_results.append(validation_result)
    train_result = np.mean(np.square(res.predict(X_train) - y_train))
    train_results.append(train_result)

    msg = (
        "%s: \nAverage CV error: %s \nStd CV Error: (%s) \nTraining Error:\n%s \n"
        % (
```

```

        name,
        str(cv_results.mean()),
        str(cv_results.std()),
        str(train_result),
        str(validation_result),
    )
)
print(msg)
print("-----")

```

Now let's compare performance of the two algorithms.

```

In [ ]: # compare algorithms
names = ["Linear Regression", "Multilinear Perceptron"]
fig = pyplot.figure()
# print('\033[1m' + 'Fig 14: Box Plot Comparing Model Performance' + '\033[0m' )
fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)
pyplot.boxplot(kfold_results)
ax.set_xticklabels(names)
fig.set_size_inches(15, 8)
pyplot.ylabel("Model Errors")
pyplot.xlabel("Models")
pyplot.suptitle(
    "Fig. 14: Box Plot Comparing Model Performance",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()

```

```

In [ ]: # compare algorithms
fig = pyplot.figure()

ind = np.arange(len(names)) # the x locations for the groups
width = 0.35 # the width of the bars

fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)
pyplot.bar(
    ind - width / 2, [x.mean() for x in train_results], width=width, label="Train Error",
)
pyplot.bar(
    ind + width / 2,
    [x.mean() for x in validation_results],
    width=width,
    label="Validation Error",
)
fig.set_size_inches(15, 8)
pyplot.legend()
pyplot.ylabel("Error")
pyplot.xlabel("Models")
ax.set_xticks(ind)
ax.set_xticklabels(names)
pyplot.suptitle(
    "Fig. 15: Bar Plot Comparing Training and Validation Error",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)

```

```
)  
pyplot.show()
```

The linear regression model has fewer errors compared to MLP despite its simplicity. To improve the performance of the MLP regressor, we seek the best parameters by use of grid search as shown below.

```
In [ ]: # Grid search : `MLPRegressor`  
        """  
        hidden_layer_sizes : tuple, length = n_layers - 2, default (100,)  
            The ith element represents the number of neurons in the ith  
            hidden layer.  
        """  
        param_grid = {"hidden_layer_sizes": [(20,), (50,), (20, 20), (20, 30, 20)]}  
        model = MLPRegressor()  
        kfold = KFold(n_splits=num_folds)  
        grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=  
        grid_result = grid.fit(X_train, y_train)  
        print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))  
        means = grid_result.cv_results_["mean_test_score"]
```

```
In [ ]: # prepare model  
        model = MLPRegressor(hidden_layer_sizes=(20, 30, 20))  
        model.fit(X_train, y_train)
```

```
In [ ]: # estimate accuracy on validation set  
        from sklearn.metrics import mean_squared_error, r2_score  
  
        predictions = model.predict(X_test)  
        mse_MLP = mean_squared_error(y_test, predictions)  
        r2_MLP = r2_score(y_test, predictions)  
  
        # prepare model  
        model_2 = LinearRegression()  
        model_2.fit(X_train, y_train)  
        predictions_2 = model_2.predict(X_test)  
  
        mse_OLS = mean_squared_error(y_test, predictions_2)  
        r2_OLS = r2_score(y_test, predictions_2)  
        print("MSE Regression = %f, MSE MLP = %f" % (mse_OLS, mse_MLP))  
        print("R2 Regression = %f, R2 MLP = %f" % (r2_OLS, r2_MLP))
```

```
In [ ]: train_size = int(len(X) * (1 - test_size))  
        X_train, X_test = X[0:train_size], X[train_size : len(X)]  
        y_train, y_test = Y[0:train_size], Y[train_size : len(X)]  
  
        modelMLP = MLPRegressor(hidden_layer_sizes=(50,))  
        modelOLS = LinearRegression()  
        model_MLP = modelMLP.fit(X_train, y_train)  
        model_OLS = modelOLS.fit(X_train, y_train)  
  
        Y_predMLP = pd.DataFrame(  
            model_MLP.predict(X_test), index=y_test.index, columns=y_test.columns  
        )  
  
        Y_predOLS = pd.DataFrame(  
            model_OLS.predict(X_test), index=y_test.index, columns=y_test.columns  
        )
```

```
model_OLS.predict(X_test), index=y_test.index, columns=y_test.columns
)
```

Let's compare the two predictions below:

```
In [ ]: pd.DataFrame(
    {
        "Actual : 1m": y_test.loc[:, "1m_pred"],
        "Prediction MLP 1m": Y_predMLP.loc[:, "1m_pred"],
        "Prediction OLS 1m": Y_predOLS.loc[:, "1m_pred"],
    }
).plot(figsize=(10, 5))
pyplot.title("1 month rate prediction")
pyplot.ylabel("Yield Rates")
pyplot.suptitle(
    "Fig. 16: 1 Month Rate Prediction",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show
```

```
In [ ]: pd.DataFrame(
    {
        "Actual : 5yr": y_test.loc[:, "5y_pred"],
        "Prediction MLP 5yr": Y_predMLP.loc[:, "5y_pred"],
        "Prediction OLS 5yr": Y_predOLS.loc[:, "5y_pred"],
    }
).plot(figsize=(10, 5))
pyplot.title("5 years rate prediction")
pyplot.ylabel("Yield Rates")
pyplot.suptitle(
    "Fig. 17: 5 Years Prediction",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show
```

```
In [ ]: pd.DataFrame(
    {
        "Actual : 30yr": y_test.loc[:, "30y_pred"],
        "Prediction MLP 30yr": Y_predMLP.loc[:, "30y_pred"],
        "Prediction OLS 30yr": Y_predOLS.loc[:, "30y_pred"],
    }
).plot(figsize=(10, 5))
pyplot.title("30 years rate prediction")
pyplot.ylabel("Yield Rates")
pyplot.suptitle(
    "Fig. 18: 30 Year Prediction",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show
```

From the charts above, the linear regression model seems to provide a better prediction of yield rates as compared to the ANN model.

5. Conclusion

In this lesson, we have applied an ordinary linear regression and a deep learning model to forecast yield curve in all short, medium, and long terms. Although the linear regression model outperformed the MLP model in this exercise, there are a variety of problems in which a deep learning model will give us superior results, and it is therefore advisable to try out a couple of models before settling on one. We have also seen that we can always improve the performance of neural networks by performing a grid search to select optimal parameters.

The output of the deep learning model can be improved by finding the right hyperparameters and increasing the layers of the model. In the next lesson, we will look at optimization algorithms in deep learning, the challenges they face, how they affect output of our models, and how to address these challenges.

References

1. Bernadell, Carlos, et al. "Yield Curve Prediction for the Strategic Investor." European Central Bank, *Working Paper Series*, no. 472, 2005.
2. Bluwstein, Kristina, et al. "Credit Growth, the Yield Curve and Financial Crisis Prediction: Evidence from a Machine Learning Approach." (2021).
3. Hyndman, Cody. "Arbitrage-Free Yield Curve and Bond Price Forecasting by Deep Neural Networks." *YouTube*, uploaded by Fields Institute, 29 Sept. 2021.
4. Arnold, Ludovic, et al. "An Introduction to Deep Learning." European Symposium on Artificial Neural Networks (ESANN), 2011.

MACHINE LEARNING IN FINANCE

MODULE 6 | LESSON 3

OPTIMIZATION

Reading Time	65 minutes
Prior Knowledge	Introduction to deep learning
Keywords	Gradient descent, stochastic gradient descent, convexity

In the first lesson, we discussed gradient descent as an optimization algorithm, used to update model parameters in order to achieve minimum loss function. In this lesson, we will study some commonly used deep learning optimization algorithms.

1. Optimization and Deep Learning

In the first lesson, we saw that given a deep learning problem, we first define a loss function then use an optimization algorithm to minimize the loss function in a dataset. While optimization's major concern is to minimize an objective function, deep learning algorithms' primary concern is to minimize the difference between our model predictions and the label we want to predict, thus allowing the neural networks to achieve better performance. This is achieved by ensuring the model does not overfit the dataset.

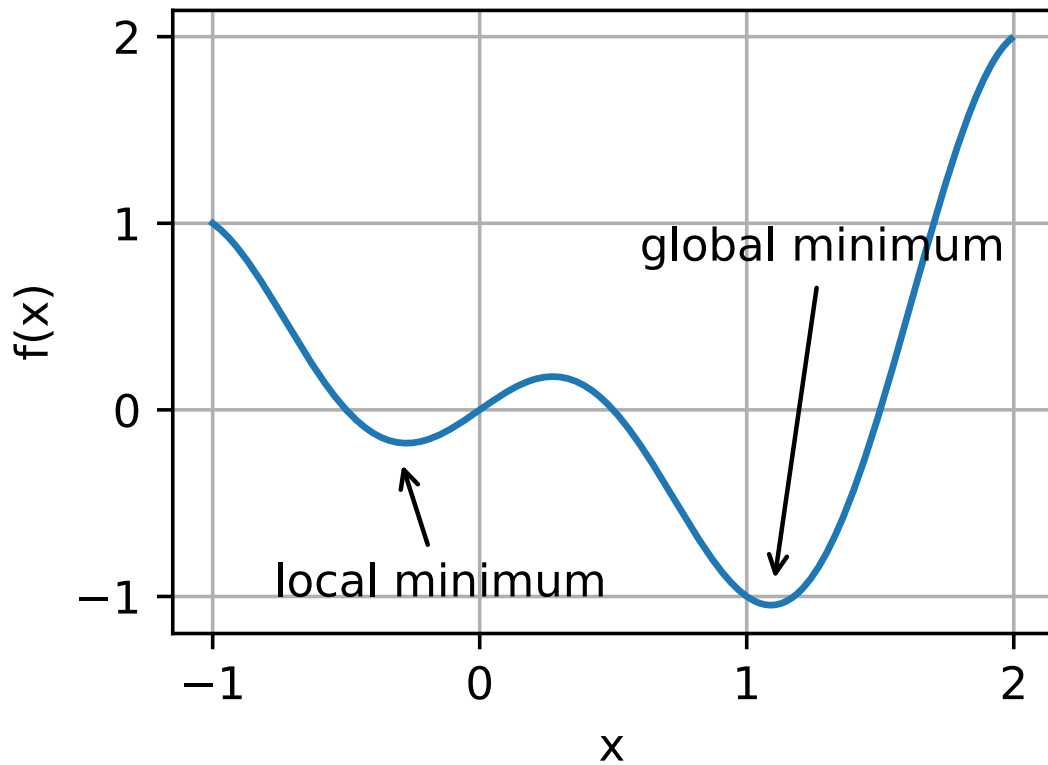
We will now discuss the challenges of optimization in deep learning.

1.1 Local Minima

Given an objective function $f(x)$, we define the **global minimum** as the value of $f(x)$ where the loss function is the minimum over the entire domain of the loss function, while a **local minimum** is found where, given a small segment of the domain, the value of $f(x)$ will be smaller at the local minimum than at any other point in the segment.

As discussed earlier, neural networks tend to be complex due to the non-linear activation functions we introduce at different layers. Therefore, the resultant loss function has more than one minimum as shown below:

Figure 1: The Difference between a Global Minimum and Local Minimum



Source: Zhang, Aston, et al. "Dive into deep learning." arXiv preprint arXiv:2106.11342 (2021).

Imagine we have a ball and our target is to slide it to the lowest point in the curve above. If we started from the left side of the curve, the ball may settle in the first trough called a local minimum. We will not have achieved our target because the ball will not have reached the lowest possible point in this curve.

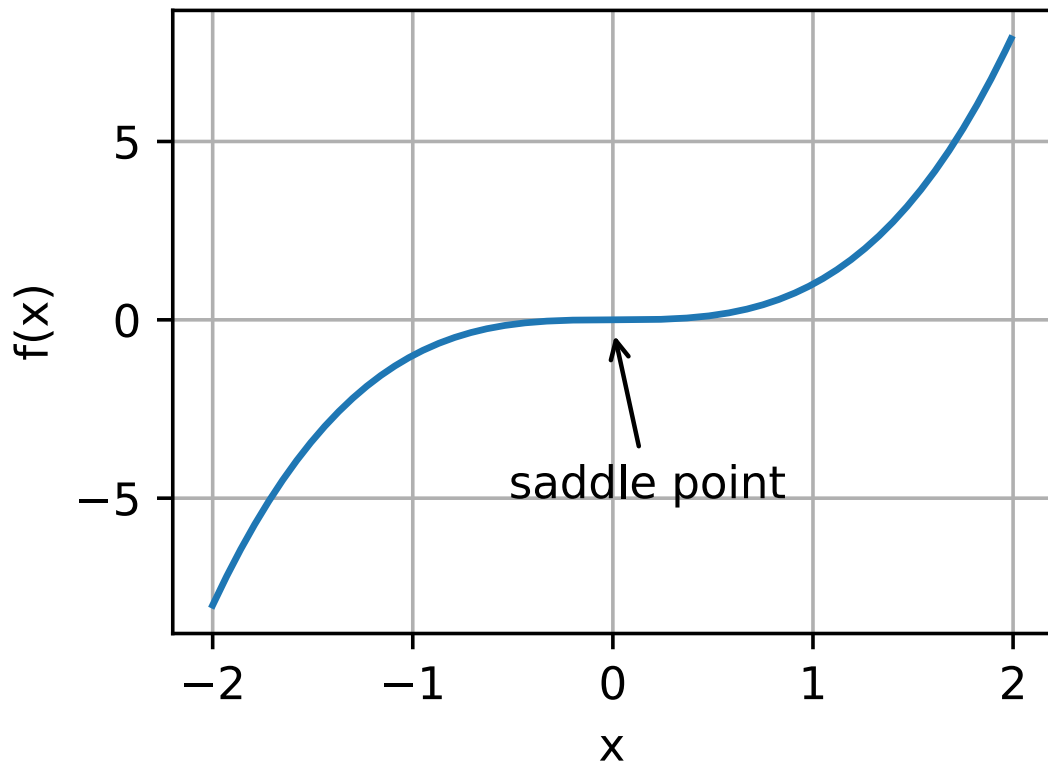
This is the scenario played by our deep learning algorithm. The best output will be achieved only when the model reaches the lowest possible point, which is called a global minimum. Otherwise, if it hits a local minimum, iterations will stop but not at an optimal point. We can escape this local minimum problem by introducing randomness in our algorithm.

A similar issue is the saddle point, which we explore next.

1.2 Saddle Points

Saddle points are points in a function where the gradient is equal to zero, yet these points are neither a minimum nor a maximum value of the loss function. As such, the optimization algorithm tends to stall at these points as the gradient is zero. Below is a diagram depicting a saddle point.

Figure 2 Saddle Point (Neither a Maximum nor a Minimum Point)



Source: Zhang, Aston, et al. "Dive into deep learning." arXiv preprint arXiv:2106.11342 (2021).

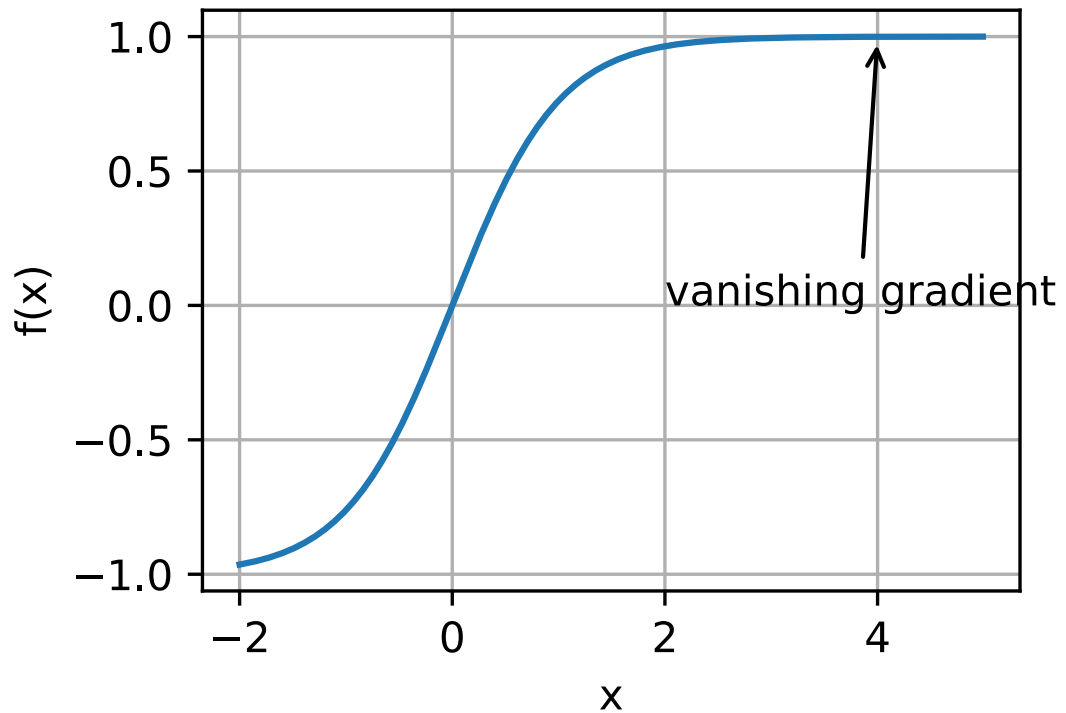
As mentioned in the previous subsection, by introducing randomness in our algorithm, we can escape the saddle points obstacle.

Finally, the other problem we face with deep learning optimization is vanishing gradients.

1.3 Vanishing Gradients

Some of the activation functions we use in deep learning tend to give us zero or values close to zero at some points. This has the unintended consequence of getting the optimization algorithm stuck at these points for a while. To handle such cases, we can reparameterize the problem. Below is an example of a vanishing gradient problem.

Figure 3: A Vanishing Gradient Scenario: A Common Problem with tanh Functions



Source: [Zhang, Aston, et al. "Dive into deep learning." arXiv preprint arXiv:2106.11342 \(2021\).](#)

In this section, we have looked at the challenges we face with optimization and how to handle them. In the next section, we are going to discuss functions that are easily solved by optimization algorithms called **convex functions**.

2. Convexity

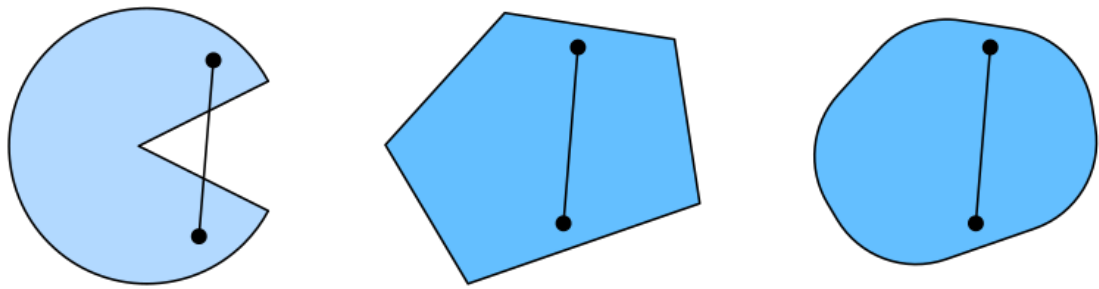
As we have seen so far, optimization algorithms play a critical role in machine learning when estimating the weights by minimizing the cost function.

Whilst studying gradient descent, we learned that the intuition behind it was to find weights that converge to a solution. These can either be a local minimum in the neighborhood or, if we are lucky, the global minimum. Understanding convexity prepares us to better understand gradient descent and generally the convergence problem.

2.1 Convex Sets

We can define a convex set as a shape in which, given any line that joins two points in this set, will never pass outside the set. See the diagrams below.

Fig. 4: Convex and Non-Convex Sets



Source: [Zhang, Aston et al. "Dive into Deep Learning."
https://d2l.ai/chapter_optimization/convexity.html

In the square and circle shapes above, the line segments drawn are well confined within the shapes, and thus the two are examples of convex sets. The diagram on the left is a non-convex set since part of the line segment passes outside the shape.

Mathematically, a convex set C is given by $\alpha x + (1-\alpha)y \in C$, $\forall x, y \in C$ and $\forall \alpha \in [0, 1]$

Next, we study convex functions, which are our main area of focus since we are interested in minima of loss functions.

2.2 Convex Functions

We say that a function f is convex if the set of points that are lying on or above the graph of the function is a convex set. Mathematically, if we have a convex set C , a function $f: C \rightarrow \mathbb{R}$ is said to be convex if for all $x, y \in C$ and $\alpha \in [0, 1]$ then $\alpha f(x) + (1-\alpha)f(y) \geq f(\alpha x + (1-\alpha)y)$.

Most of the cost functions we will encounter are non-convex and we may need to test for the convexity of the function. Below are techniques we use to test whether a function is convex or not.

- A convex function has its second order derivative greater than or equal to zero, that is, $f''(x) \geq 0$. Examples are $y = e^x$ and $y = x^2$.
- If $-f''(x) \geq 0$, we call this function concave.

From the above discussion it is clear that the local minimum of a convex function is the same as its global minimum.

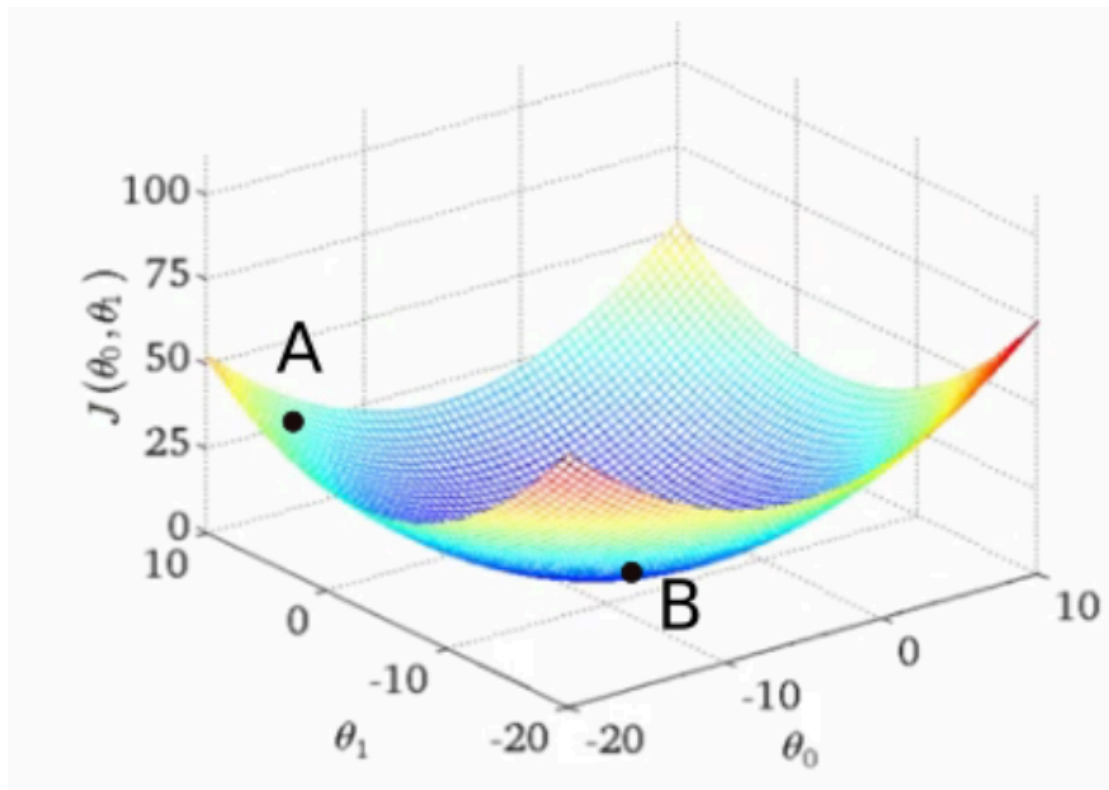
In the next section, we seek to understand the role of convexity in gradient descent algorithms.

3. Gradient Descent

In the first lesson, when training a neural network, we assigned initial weights randomly, and as such, we expect not to get accurate predictions. The aim of gradient descent was to assist us in getting the loss function at a minimum.

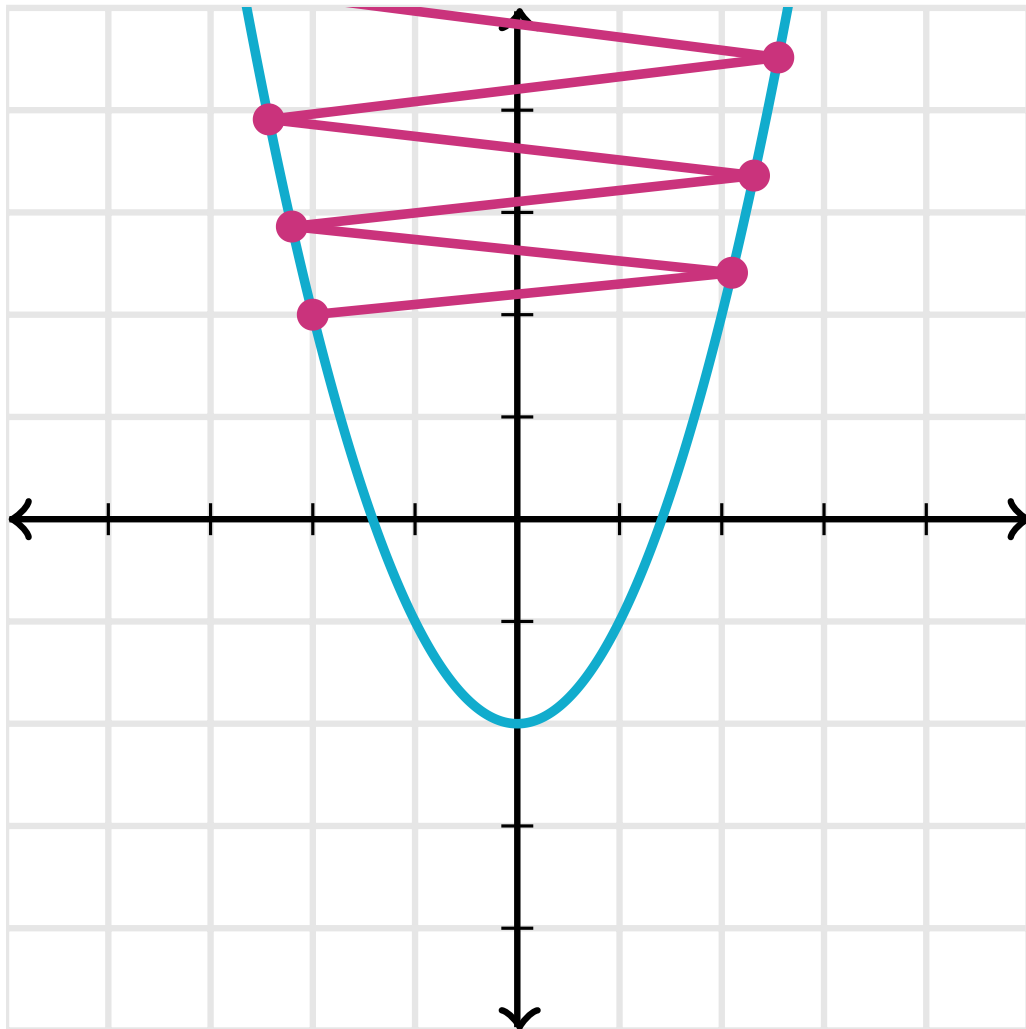
Once we initialize the weights and we are at a point A in the loss curve below, we first check the direction that gives us the steepest decline in the loss function value. We take this direction, which is the direction opposite to the gradient's direction and it gives the direction with the steepest ascent.

Fig. 5: Gradient Descent



Now that we have the direction of the steepest ascent, we then choose the size of step to be taken in this direction. We call this step a **learning rate**. The learning rate must be chosen carefully to take us to the minimum. Large learning rates may cause the model to overshoot the minima as it bounces along the slopes of the function and never reaches the minimum, as in the path shown below.

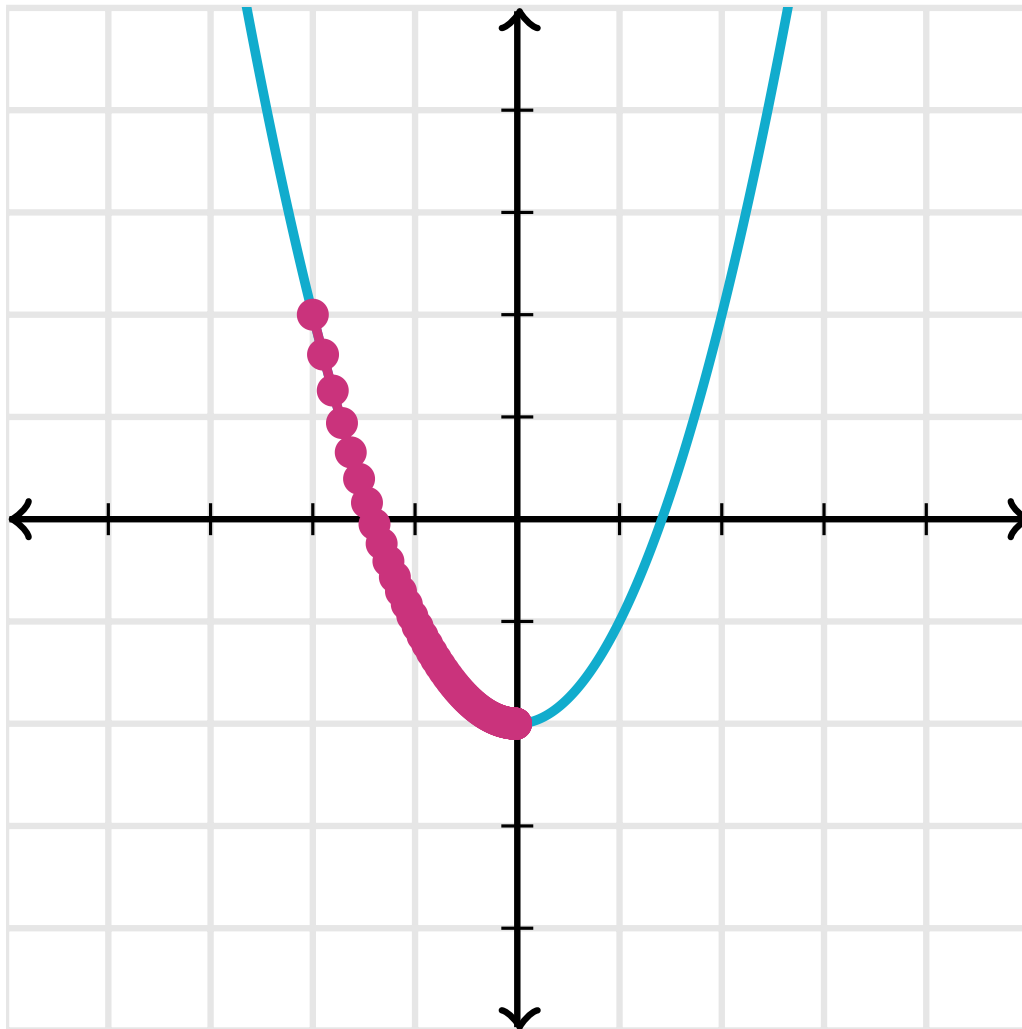
Fig. 6 A Plot of the Problem with Large Steps in Gradient Steps



Source: Khan Academy. "Gradient Descent."
(<https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>)

On the other hand, a small learning rate will make the model too slow, thus making the process infeasible. Another challenge with a small learning rate is that the model becomes stagnant toward a local minimum.

Fig. 7 Gradient Steps with Small Step Sizes



Source: Khan Academy. "Gradient Descent."
<https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>

Once we get both the gradient and learning rate, the next step is to calculate the gradient at our new point in the loss curve and repeat the process iteratively.

The gradient not only gives us the steepest ascent direction, its magnitude tells us how steep the ascent/descent will be. At the base of the curve, we expect the gradient to be approximately zero. Note that in practice we rarely reach the minima, but we get to a flat region where the loss function is closest to the minimum that can be achieved, and even after many iterations, the loss values don't improve much. As such, we say our model has fully converged.

The equation that describes the above updated rule of gradient descent is:
$$w \leftarrow w - \alpha \times \nabla_w \sum_{i=1}^m L_m(w) \quad (1)$$

We perform this update in every iteration where w represents the weight's vector, and it is from this vector that we subtract the gradient of the loss function multiplied by the learning rate α .

For better performance of the gradient descent, a widely adapted technique by practitioners is to have a varying learning rate where we start with a large learning rate, and as we approach the minimum, we use a smaller learning rate. This approach is referred to as **simulated annealing**, and we will discuss more of it in the computational finance course.

As mentioned before, optimization algorithms—which gradient descent is a part of—suffer some challenges such as local minima and saddle points. We also mentioned that we can overcome these challenges by introducing randomness in our algorithm.

Stochastic gradient descent (SGD) is an example of a stochastic gradient descent algorithm in which random noise has been added. Let us see how it works.

4. Stochastic Gradient Descent

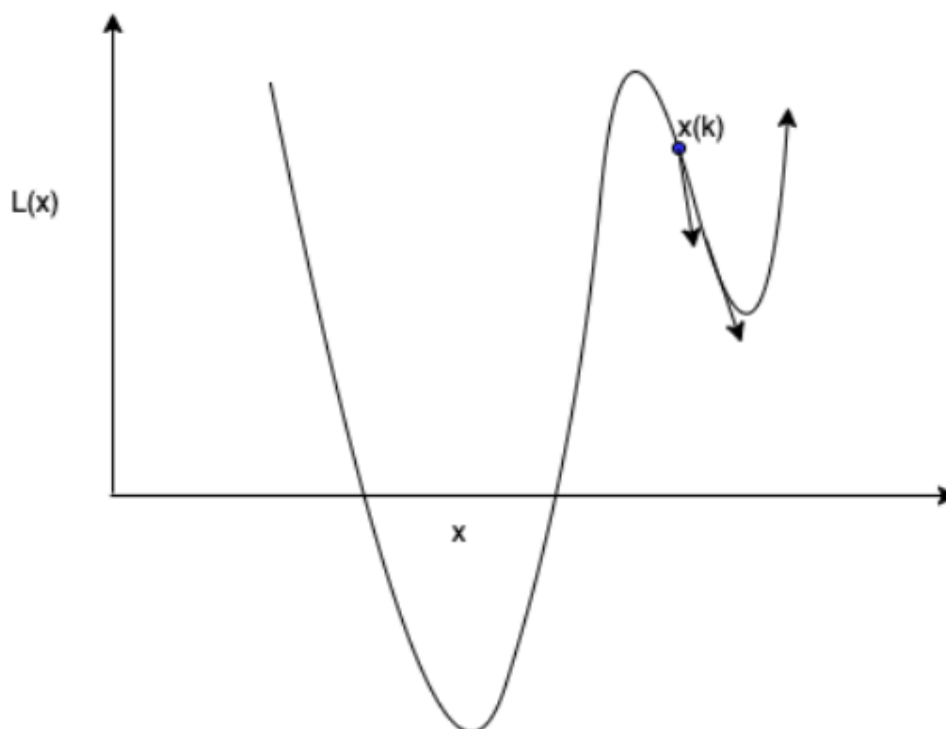
In the previous section, we saw that gradient descent updates the weights using the algorithm below:

```
for k in [1, number_iterations]:
```

```
     $X[k+1] = X[k] - \alpha * \text{gradient } L[X(k)]$ 
```

The limitation of this is, with a good choice of α , it will work for convex functions, but that does not necessarily mean that they will generalize well. For the algorithm to provide better generalization, they should be able to work with non-convex functions as well. As seen earlier, if the loss function is non-convex and our initial weight guess is not close to the global minima, then the gradient descent would converge at a local minima.

Fig. 8 A Demonstration of the Stochastic Gradient Descent



Stochastic gradient descent allows us to pick random samples in every iteration where we calculate the loss as shown below:

```
for k in [1, number_iterations]:  
  
    pick a random datapoint  $d_i$ ;  
  
    calculate loss  $L_i$   
  
     $X[k+1] = X[k] - \alpha * \text{gradient } L_i[X(k)]$ 
```

We expect the initial iterations of SGD to have a lot of noise; hence, the gradient will behave very erratically. This random loss behavior can allow the steps to escape the local minima curse. Other advantages of SGD over gradient descent are:

- SGD converges faster than gradient descent.
- SGD is convenient for training data coming in as streams.
- When we have very large datasets that can not be processed by RAM, SGD allows us to process smaller datasets at each iteration.

Another variant of the SGD that is commonly used is the **minibatch SGD**, which we ask students to study in the reference provided.

5. Implementing Gradient Descent

Gradient descent works well with convex functions. If f is the convex function we would like to minimize, then at each iteration, our goal will be to get $f(x_{t+1}) \leq f(x_t)$. Therefore, gradient descent gradually computes the minimum of the cost function.

Below is a Python code implementation of gradient descent with a linear regression problem.

```
In [ ]: # Importing Libraries  
import matplotlib.pyplot as plt  
import numpy as np
```

We first define the cost function, in this case the mean squared error, that gives the difference between the predicted value and the actual value given by the equation
$$\text{Cost} = \frac{1}{n} \sum_1^n (y - \hat{y})^2$$

```
In [ ]: def MSE(y_true, y_predicted):  
    # Calculating the loss or cost defined as MSE  
    return np.sum((y_predicted - y_true) ** 2) / len(y_true)
```

We then define the gradient function below. The hyperparameters we consider are the number of iterations and learning rate, and we define the stopping threshold that will stop the iterations once we hit the threshold. The derivative of the cost function with respect to the weight is given by:
$$\frac{\partial \text{cost}}{\partial W} = -\frac{2}{n} \sum_1^n x(y - \hat{y})$$

and the derivative of the cost function with respect to bias is $\frac{\partial}{\partial b} \text{cost} = -\frac{2}{n} \sum_1^n (y - \hat{y})$

```
In [ ]: def gradient_descent_algorithm(
    x, y, iterations=10000, alpha=0.0001, stopping_threshold=1e-7
):
    # Initializing weight, bias, learning rate and iterations
    weight = 0.1
    bias = 0.01
    iterations = iterations
    learning_rate = alpha
    n = float(len(x))

    costs = []
    weights = []
    previous_cost = None

    # Estimation of optimal parameters
    for i in range(iterations):
        # Making predictions
        y_predicted = (weight * x) + bias

        # Calculating the current cost
        cost = MSE(y, y_predicted)

        # If the change in cost is less than or equal to
        # stopping_threshold we stop the gradient descent
        if previous_cost and abs(previous_cost - cost) <= stopping_threshold:
            break

        previous_cost = cost

        costs.append(cost)
        weights.append(weight)

        # Calculating the gradients
        weight_derivative = -(2 / n) * sum(x * (y - y_predicted))
        bias_derivative = -(2 / n) * sum(y - y_predicted)

        # Updating weights and bias
        weight = weight - (learning_rate * weight_derivative)
        bias = bias - (learning_rate * bias_derivative)

        # Printing the parameters for each 1000th iteration
        # print(f"Iteration {i+1}: Cost {cost}, Weight \
        # {weight}, Bias {bias}")

    # Visualizing the weights and cost at for all iterations
    plt.figure(figsize=(8, 6))
    plt.plot(weights, costs)
    plt.scatter(weights, costs, marker="o", color="red")
    plt.title("Cost vs Weights")
    plt.suptitle(
        "Fig. 9: Cost vs Weights",
        fontweight="bold",
        horizontalalignment="right",
        fontsize=15,
    )
    plt.ylabel("Cost")
```

```
plt.xlabel("Weight")
plt.show()

return weight, bias, cost
```

Below, we consider an example with the data:

```
In [ ]: # Data
X = np.array(
    [
        3.66192704,
        0.4367313,
        3.86850312,
        0.28627004,
        0.64033297,
        1.84354183,
        3.51648513,
        2.33171731,
        2.40850115,
        1.0959004,
        3.24136739,
        0.39645735,
        2.44789746,
        0.01160215,
        2.02542692,
        0.19118386,
        2.07730598,
        0.10467331,
        0.50427978,
        3.04388398,
        2.01930518,
        2.5796251,
        2.47759331,
        3.86474532,
        2.51074925,
        2.8197363,
        1.83506425,
        1.01451347,
        1.48428389,
        1.0218188,
        2.85357911,
        1.54562156,
        0.04014026,
        0.02950905,
        1.9922938,
        2.48649378,
        0.20470277,
        2.88649838,
        3.02218141,
        1.62948796,
        2.46788939,
        1.26114926,
        0.74808066,
        0.95436498,
        2.26441095,
        1.00789754,
        1.18732863,
        3.91176711,
        1.0266336,
```

```

2.32570421,
2.80067965,
0.83060144,
3.85524446,
2.50885513,
3.04304448,
1.90918746,
2.80970553,
1.97939314,
0.26492414,
2.16720092,
3.74822112,
2.72372863,
2.6020474,
2.61888224,
0.02828909,
1.65610441,
0.41891066,
3.36760303,
2.0998768,
1.98957548,
1.43377349,
2.22304239,
0.24350151,
2.52882744,
0.25768373,
0.85605071,
2.3646854,
3.09164147,
2.87290517,
2.83387608,
0.30647763,
0.68189712,
0.38868718,
2.23428206,
3.74299469,
1.54474145,
1.08338491,
3.96838634,
0.98634429,
3.03813934,
1.45576515,
3.25167005,
1.11292405,
3.71206365,
2.31440203,
3.16539218,
0.87050119,
3.04915131,
3.16137918,
2.05815803,
]
)
Y = np.array(
[
-3.33807296,
-6.5632687,
-3.13149688,
-6.71372996,
-6.35966703,

```

-5.15645817,
-3.48351487,
-4.66828269,
-4.59149885,
-5.9040996,
-3.75863261,
-6.60354265,
-4.55210254,
-6.98839785,
-4.97457308,
-6.80881614,
-4.92269402,
-6.89532669,
-6.49572022,
-3.95611602,
-4.98069482,
-4.4203749,
-4.52240669,
-3.13525468,
-4.48925075,
-4.1802637,
-5.16493575,
-5.98548653,
-5.51571611,
-5.9781812,
-4.14642089,
-5.45437844,
-6.95985974,
-6.97049095,
-5.0077062,
-4.51350622,
-6.79529723,
-4.11350162,
-3.97781859,
-5.37051204,
-4.53211061,
-5.73885074,
-6.25191934,
-6.04563502,
-4.73558905,
-5.99210246,
-5.81267137,
-3.08823289,
-5.9733664,
-4.67429579,
-4.19932035,
-6.16939856,
-3.14475554,
-4.49114487,
-3.95695552,
-5.09081254,
-4.19029447,
-5.02060686,
-6.73507586,
-4.83279908,
-3.25177888,
-4.27627137,
-4.3979526,
-4.38111776,
-6.97171091,

```

-5.34389559,
-6.58108934,
-3.63239697,
-4.9001232,
-5.01042452,
-5.56622651,
-4.77695761,
-6.75649849,
-4.47117256,
-6.74231627,
-6.14394929,
-4.6353146,
-3.90835853,
-4.12709483,
-4.16612392,
-6.69352237,
-6.31810288,
-6.61131282,
-4.76571794,
-3.25700531,
-5.45525855,
-5.91661509,
-3.03161366,
-6.01365571,
-3.96186066,
-5.54423485,
-3.74832995,
-5.88707595,
-3.28793635,
-4.68559797,
-3.83460782,
-6.12949881,
-3.95084869,
-3.83862082,
-4.94184197,
    ]
)

# Using the gradient descent algorithm to find approximated weight and bias
weight, bias, cost = gradient_descent_algorithm(
    X, Y, iterations=1000, stopping_threshold=1e-5
)
print(f"Estimated Weight: {weight}\nEstimated Bias: {bias}")

# Making predictions using estimated parameters
Y_pred = weight * X + bias

# # Plotting the regression line
# plt.figure(figsize = (8,6))
# plt.scatter(X, Y, marker='o', color='red')
# plt.plot([min(X), max(X)], [min(Y_pred), max(Y_pred)], color='blue',markerface
#         markersize=10,linestyle='dashed')
# `plt.xlabel`("X")
# `plt.ylabel`("Y")
# plt.show()

```

```

In [ ]: plt.figure(figsize=(8, 6))
plt.scatter(X, Y, marker="o", color="red")
plt.plot(
    [min(X), max(X)],

```

```
[min(Y_pred), max(Y_pred)],  
color="blue",  
markerfacecolor="red",  
markersize=10,  
linestyle="dashed",  
)  
plt.xlabel("X")  
plt.ylabel("Y")  
plt.suptitle(  
    "Fig. 10: Linear Regression Prediction",  
    fontweight="bold",  
    horizontalalignment="right",  
    fontsize=15,  
)  
plt.show()
```

6. Conclusion

In this lesson, we have covered optimization in deep learning and extensively explored the gradient descent algorithm. We have also seen how easy it is to implement gradient descent.

Now that we have introduced deep learning, in our next lesson, we will study an application of deep learning in predicting credit risk.

References

- Khan Academy. "Gradient Descent."
<https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>.
- Zhang, Aston et al. "Dive into Deep Learning." *arXiv* preprint arXiv:2106.11342, 2021,
<https://d2l.ai/d2l-en.pdf>.

MACHINE LEARNING IN FINANCE

MODULE 6 | LESSON 4

CREDIT RISK MODELING: APPLICATION OF DEEP LEARNING

Reading Time	120 minutes
Prior Knowledge	Introduction to machine learning, introduction to deep learning
Keywords	Expected Loss, credit risk, SMOTE

In the previous lessons of this module, we have built on the theory of deep learning. In this lesson, we will discuss one of the hottest topics in machine learning, that is, predicting loan default. We will use classical machine learning algorithms on a refined dataset and discuss the steps used to predict mortgage default.

1. Mortgage Delinquency

There has been an evolution in the business of lending money as the process has become increasingly complex due to the growing market demands and clients' increasing appetite for credit. These factors among others have led to an increase in regulation and oversight in the banking industry so as to make sure they act responsibly when issuing loans. In the recent past, the rate of digitalization globally has shot up with people in remote parts of the world having access to phones. This has made it possible for people to use mobile devices as a financial medium through which they can send and receive money to and from other people around the world. These transactions happen in a matter of seconds. Many fintechs have taken advantage of this to launch microloans to customers who are low risk. The fintechs use the interaction of the customers with their gadgets to build a credit score for each of the customers and determine the probability of the customer defaulting on a loan. This logic also applies to mortgages and the probability of mortgage default is called mortgage delinquency. Machine learning models are trained on the data we have processed and the decision making process of giving loans is automated.

The machine learning models are used to assess the creditworthiness of a borrower. Before the advent of machine learning, lenders had an established guideline to measure creditworthiness. These guidelines were based on the five C's listed below:

1. Character that looks at the borrower's repayment and credit record.

2. Capacity that assess the borrower's ability to service the loan by looking at the debt-to-income ratio.
3. Capital that looks at the down payment the borrower has paid. This is used to determine how serious the borrower is.
4. Collateral, which is the asset provided to secure the mortgage, such as another home.
5. Conditions of the borrower's environment, like the state of the economy.

However, this has posed serious challenges to lenders as the number of features are limited in assessing customers' creditworthiness, with potentially credit-worthy clients being denied credit for failing certain criteria, and their inability to keep pace with the technological evolution that has been witnessed in the past decade.

It is because of these limitations that machine learning models are now at the heart of assessing the creditworthiness of borrowers. However, recent research has shown that deep learning has the potential to eclipse machine learning for assessing credit risks. Deep neural networks are great at detecting risky borrowers when the data is unstructured and very complex.

However, the risk of using deep learning models is that in most cases they are not explainable, that is, they are like black boxes and we are in most cases unable to know what happened for us to get a certain output. Currently, a lot of research is being done to address this by focusing on explainable artificial intelligence. In the next section, we are going to show a simple example of using a classical machine learning algorithm to assess credit risk.

2.Credit Risk

2.1 Data Sources

We downloaded our data from the [UCI marketing](#) dataset, and we did some engineering and cleaning to make the data ready for model training.

In the cell below, we import packages we will use in this demonstration.

```
In [2]: # !pip install imbalanced-Learn
```

```
In [3]: from collections import Counter

# EDA
import matplotlib.pyplot as plt
import numpy as np

# data manipulation
import pandas as pd
import seaborn as sns
from imblearn.combine import SMOTETomek
from scipy import stats

# feature selection
```

```

from sklearn.ensemble import RandomForestClassifier

# algorithms
from sklearn.linear_model import LogisticRegression

# model evaluation
from sklearn.metrics import (
    accuracy_score,
    brier_score_loss,
    classification_report,
    cohen_kappa_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)

# machine learning
from sklearn.model_selection import train_test_split

```

2.2 Loading the Dataset

We start by uploading this downloaded data and taking a rough look at the data. In this problem, we will skip the process of performing EDA and data preprocessing techniques as the data is already processed.

- Person Age, Person Income, Person Employment Length, Loan Amount, Loan interest rate, Loan percent income and Person credit history length were transformed using min max scaler, that is, converting the values to range between 0 and 1.
- The other columns were categorical and we therefore converted them to numerical data by one hot encoding the columns.

In [4]: `df = pd.read_csv("credit_sample.csv")`

In [5]: `df.head()`

Out[5]:

	loan_status	person_age	person_income	person_emp_length	loan_amnt	loan_int_rate
0	0	0.015625	0.002751	0.121951	0.014493	0.32134
1	1	0.078125	0.002751	0.024390	0.144928	0.41853
2	1	0.046875	0.030209	0.097561	1.000000	0.55112
3	1	0.062500	0.024757	0.195122	1.000000	0.49719
4	1	0.015625	0.002898	0.048780	0.057971	0.09662

5 rows × 27 columns



In [6]: `df.shape`

Out[6]: (29459, 27)

```
In [7]: df.dtypes
```

```
Out[7]: loan_status          int64
person_age          float64
person_income        float64
person_emp_length    float64
loan_amnt            float64
loan_int_rate         float64
loan_percent_income   float64
cb_person_cred_hist_length float64
person_home_ownership_MORTGAGE int64
person_home_ownership_OTHER int64
person_home_ownership_OWN int64
person_home_ownership_RENT int64
loan_intent_DEBTCONSOLIDATION int64
loan_intent_EDUCATION int64
loan_intent_HOMEIMPROVEMENT int64
loan_intent_MEDICAL int64
loan_intent_PERSONAL int64
loan_intent_VENTURE int64
loan_grade_A int64
loan_grade_B int64
loan_grade_C int64
loan_grade_D int64
loan_grade_E int64
loan_grade_F int64
loan_grade_G int64
cb_person_default_on_file_N int64
cb_person_default_on_file_Y int64
dtype: object
```

```
In [8]: df.loan_status.value_counts()
```

```
Out[8]: loan_status
0      22996
1       6463
Name: count, dtype: int64
```

Clearly, the data are highly imbalanced.

```
In [9]: # plots count
ax = sns.countplot(x=df["loan_status"])

# sets the figure size in inches
ax.figure.set_size_inches(12, 6)

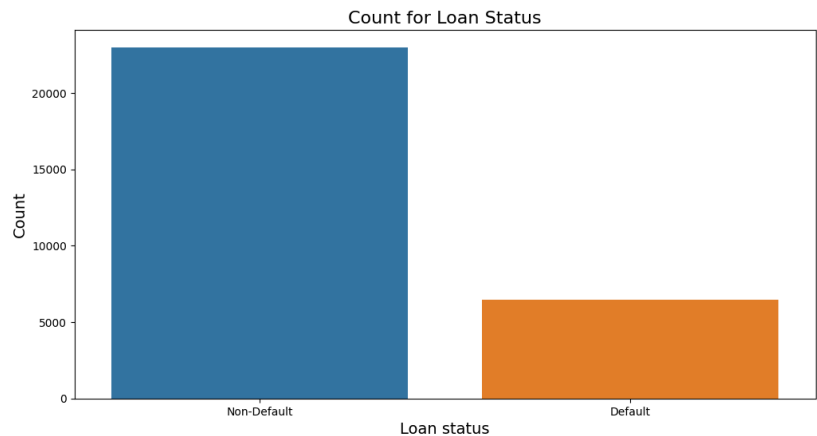
# set plot features
ax.set_title("Count for Loan Status", fontsize=16)
ax.set_ylabel("Count", fontsize=14)
ax.set_xlabel("Loan status", fontsize=14)

# set `xticks` labels
plt.xticks([0, 1], ["Non-Default", "Default"])

plt.suptitle(
    "Fig. 1: Top: A bar plot of good versus bad loans. The graph shows that the",
    fontweight="bold",
    horizontalalignment="right",
)
```

```
# displays plot
plt.show()
```

Fig. 1: Top: A bar plot of good versus bad loans. The graph shows that the dataset is highly imbalanced.



```
In [10]: # separating the data set for easier analysis
df_default = df[df["loan_status"] == 1].copy()
df_non_default = df[df["loan_status"] == 0].copy()

# counts the number of defaults and non-defaults
total_default = df_default.shape[0]
total_non_default = df_non_default.shape[0]
total_loans = df.shape[0]

print("Number of default cases:", total_default)
print(
    "This is equivalent to {:.2f}% of the total loans".format(
        (total_default / total_loans) * 100
    )
)

print("\nNumber of non-default cases:", total_non_default)
print(
    "This is equivalent to {:.2f}% of the total loans".format(
        (total_non_default / total_loans) * 100
    )
)
```

Number of default cases: 6463

This is equivalent to 21.94% of the total loans

Number of non-default cases: 22996

This is equivalent to 78.06% of the total loans

2.3 Split Dataset into Train and Test

```
In [11]: # creates the X and y data sets
X = df.drop("loan_status", axis=1).values
y = df["loan_status"].values

# splits into training and test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=2022, stratify=y
)
```

2.4 Balancing Data for Training

Since we saw that the data was highly imbalanced, we performed oversampling with synthetic minority over-sampling technique (SMOTE) with the package below.

```
In [12]: # counts the number of classes before oversampling
print("Before balancing:", Counter(y_train))

# defines the resampler
resampler = SMOTETomek(random_state=2022, n_jobs=-1)

# transforms the data set
X_balanced, y_balanced = resampler.fit_resample(X_train, y_train)

# counts the number of classes after oversampling
print("After balancing:", Counter(y_balanced))
```

Before balancing: Counter({0: 17247, 1: 4847})

After balancing: Counter({1: 16988, 0: 16988})

```
In [13]: # plots before and after SMOTE
plt.figure(figsize=(12, 4))

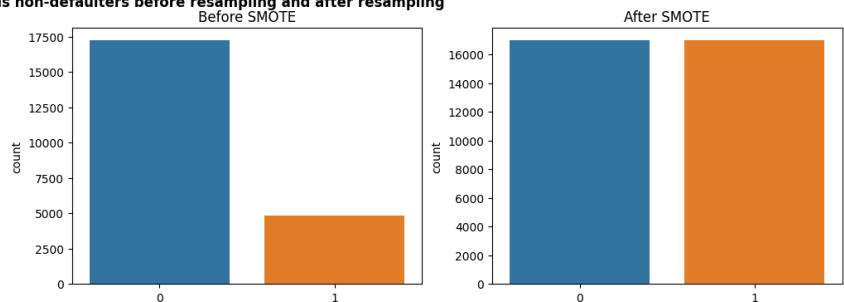
plt.subplot(1, 2, 1)
sns.countplot(x=y_train)
plt.title("Before SMOTE")

plt.subplot(1, 2, 2)
sns.countplot(x=y_balanced)
plt.title("After SMOTE")

plt.suptitle(
    "Fig. 2: Top: A plot of the defaulters versus non-defaulters before resampling",
    fontweight="bold",
    horizontalalignment="right",
)

plt.show()
```

Fig. 2: Top: A plot of the defaulters versus non-defaulters before resampling and after resampling



```
In [14]: print("Total records BEFORE:", X_train.shape[0])
print("Total records AFTER:", X_balanced.shape[0])
print("Difference =", X_train.shape[0] - X_balanced.shape[0])
```

Total records BEFORE: 22094

Total records AFTER: 33976

Difference = -11882

We now see that both the bad and good target variable are equal. Why do we need to balance our dataset?

2.5 Training the Models and Getting the Performance Metrics

As the process of model building affects the business, a discussion with business owners on the objective of the model will highly affect how we set thresholds. If the purpose is customer acquisition at the expense of risk, we can always lower the threshold of probability so as to get as many customers as possible. If we are interested in optimizing to get high profits or minimize high risks, then we can increase the threshold.

```
In [15]: # Set the threshold for defaults
THRESHOLD = 0.50
```

In this exercise, we will use two classifiers, logistic regression that is highly used by banks to discriminate between defaulters and non-defaulters, because it is easy to understand how it works.

```
In [16]: # list of classifiers
classifiers = [
    LogisticRegression(max_iter=220, random_state=2022),
    RandomForestClassifier(random_state=2022),
]
```

The function below trains the models on a dataset and evaluates their performance on unseen data.

```
In [17]: def calculate_model_metrics(model, X_test, y_test, model_probs, threshold):
    """
    Calculates Accuracy, F1-Score, PR AUC
    """
    # keeps probabilities for the positive outcome only
    probs = pd.DataFrame(model_probs[:, 1], columns=["prob"])

    # applies the threshold
    y_pred = probs["prob"].apply(lambda x: 1 if x > threshold else 0)

    # calculates f1-score
    f1 = f1_score(y_test, y_pred)

    # calculates accuracy
    accuracy = accuracy_score(y_test, y_pred)

    # calculates kappa score
    kappa = cohen_kappa_score(y_test, y_pred)

    # calculates AUC
    auc_score = roc_auc_score(y_test, probs)

    # calculates the precision
    precision = precision_score(y_test, y_pred)

    # calculates the recall
```

```

recall = recall_score(y_test, y_pred)

return accuracy, kappa, f1, auc_score, precision, recall

```

```

In [18]: def get_classifiers_performance(
X_train, X_test, y_train, y_test, threshold, classifiers
):
    # creates empty data frame
    df_performance = pd.DataFrame()

    for clf in classifiers:
        print("Training " + type(clf).__name__ + "...")
        # fits the classifier to training data
        clf.fit(X_train, y_train)

        # predict the probabilities
        clf_probs = clf.predict_proba(X_test)

        # calculates model metrics
        (
            clf_accuracy,
            clf_kappa,
            clf_f1,
            clf_auc,
            clf_precision,
            clf_recall,
        ) = calculate_model_metrics(clf, X_test, y_test, clf_probs, threshold)

        # creates a dict
        clf_dict = {
            "model": [type(clf).__name__, "---"],
            "precision": [clf_precision, np.nan],
            "recall": [clf_recall, np.nan],
            "f1-Score": [clf_f1, np.nan],
            "ROC AUC": [clf_auc, np.nan],
            "accuracy": [clf_accuracy, np.nan],
            "cohen kappa": [clf_kappa, np.nan],
        }

        # concatenate Data Frames
        df_performance = pd.concat([df_performance, pd.DataFrame(clf_dict)])

    # resets Data Frame index
    df_performance = df_performance.reset_index()

    # drops index
    df_performance.drop("index", axis=1, inplace=True)

    # gets only the odd numbered rows
    rows_to_drop = np.arange(1, len(classifiers) * 2, 2)

    # drops unwanted rows that have no data
    df_performance.drop(rows_to_drop, inplace=True)

    # returns performance summary
    return df_performance

```

```

In [19]: # calculates classifiers performance
df_performances = get_classifiers_performance(

```

```

X_balanced, X_test, y_balanced, y_test, THRESHOLD, classifiers
)
# highlight max values for each column
df_performances.style.highlight_max()

```

Training LogisticRegression...

Training RandomForestClassifier...

Out[19]:

	model	precision	recall	f1-Score	ROC AUC	accuracy	cohen kappa
0	LogisticRegression	0.555309	0.773515	0.646496	0.871806	0.814392	0.525213
2	RandomForestClassifier	0.897513	0.737005	0.809378	0.927522	0.923829	0.762356

Clearly, the random forest classifier outperforms logistic regression in nearly all the metrics with the exception of recall.

2.6 Probability Distribution

```

In [20]: # instantiates the classifiers
lr_clf = LogisticRegression(max_iter=220, random_state=2022)
rf_clf = RandomForestClassifier(random_state=2022)

# trains the classifiers
lr_clf.fit(X_balanced, y_balanced)
rf_clf.fit(X_balanced, y_balanced)

# store the predicted probabilities for class 1
y_pred_lr_prob = lr_clf.predict_proba(X_test)[:, 1]
y_pred_rf_prob = rf_clf.predict_proba(X_test)[:, 1]

```

```

In [21]: # sets plot size
plt.figure(figsize=(10, 6))

# plots
sns.kdeplot(y_pred_lr_prob, label="Logistic Regression")
sns.kdeplot(y_pred_rf_prob, label="Random Forest")

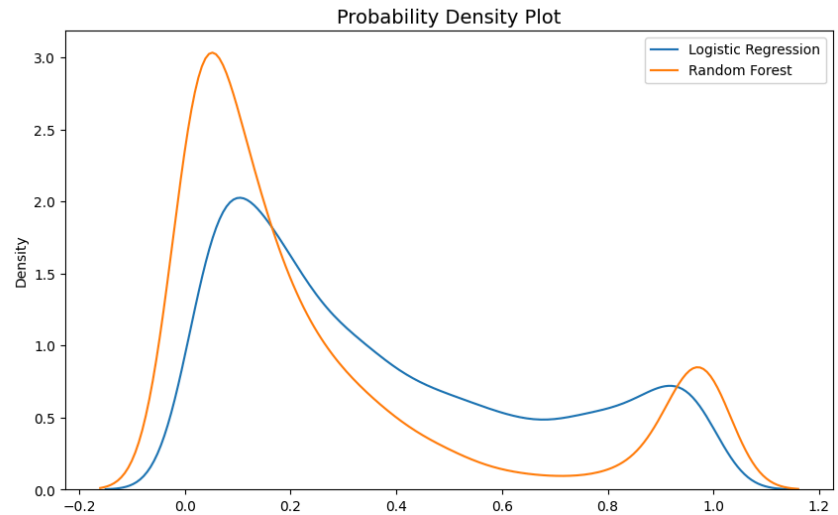
# sets the plot features
plt.title("Probability Density Plot", fontsize=14)
plt.legend()

plt.suptitle(
    "Fig. 3: Top: A probability density plot for the predicted performance of cu
    fontweight="bold",
    horizontalalignment="right",
)

# displays the plot
plt.show()

```

Fig. 3: Top: A probability density plot for the predicted performance of customers



We can observe in the density plot above that the largest concentration of probabilities is around 0, and logistic regression has moderate distributed probabilities.

```
In [22]: # set axes
fig, ax = plt.subplots()

# set fig size
ax.figure.set_size_inches(12, 8)

# plot probability - Logistic Regression
plt.subplot(2, 2, 1)
ax = stats.probplot(y_pred_lr_prob, plot=plt)

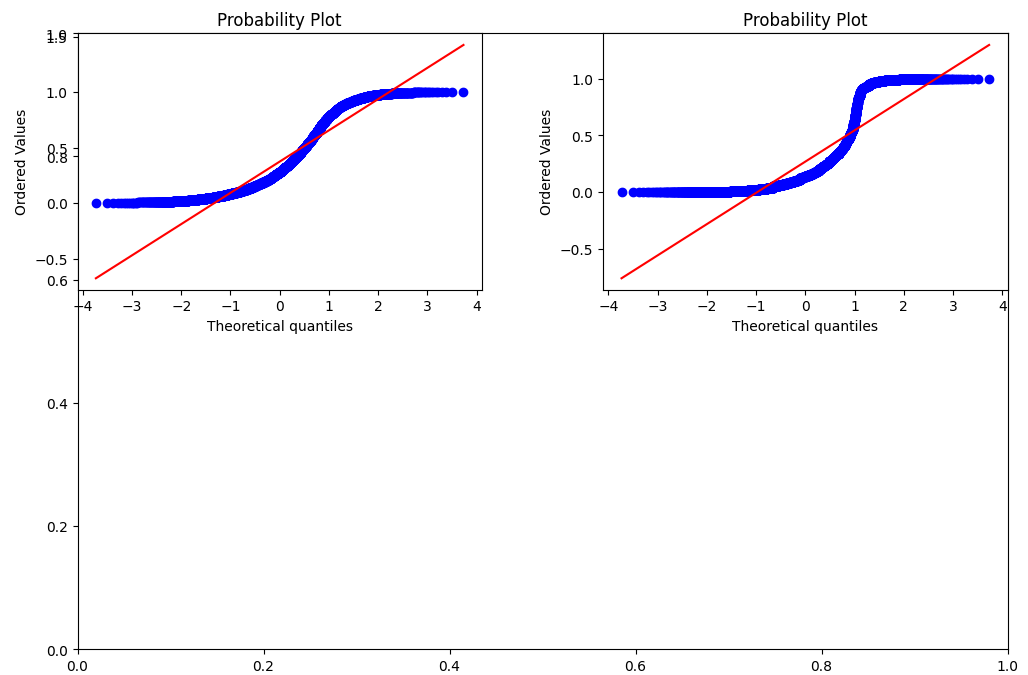
# plot probability - Random Forest
plt.subplot(2, 2, 2)
ax = stats.probplot(y_pred_rf_prob, plot=plt)

plt.subplots_adjust(wspace=0.3)
plt.subplots_adjust(hspace=0.4)

plt.suptitle(
    "Fig. 4: Top: Comparing model performance and theoretical expectation",
    fontweight="bold",
    horizontalalignment="right",
)

# displays the plot
plt.show()
```

Fig. 4: Top: Comparing model performance and theoretical expectation



The data seem to follow the theoretical distribution when we use the two models.

```
In [23]: # calculates the Brier Score Loss
bsl_lr = brier_score_loss(y_test, y_pred_lr_prob, pos_label=1)
bsl_rf = brier_score_loss(y_test, y_pred_rf_prob, pos_label=1)

# prints the calculated Brier Score Loss for each algorithm probability
print(f"Brier Score Loss (Logistic Regression): {np.round(bsl_lr, 2)}")
print(f"Brier Score Loss (Random Forest): {np.round(bsl_rf, 2)}")
```

Brier Score Loss (Logistic Regression): 0.14

Brier Score Loss (Random Forest): 0.07

The smaller the value of the Brier score, the better. The Brier score is made up of refinement loss and calibration loss. Clearly, we can see that random forest performs better since it give us a lower Brier score.

```
In [24]: # makes predictions
y_pred_lr = lr_clf.predict(X_test)
y_pred_rf = rf_clf.predict(X_test)

print("Classification Report for " + type(lr_clf).__name__)
print(classification_report(y_test, y_pred_lr))

print("\nClassification Report for " + type(rf_clf).__name__)
print(classification_report(y_test, y_pred_rf))
```

Classification Report for LogisticRegression					
	precision	recall	f1-score	support	
0	0.93	0.83	0.87	5749	
1	0.56	0.77	0.65	1616	
accuracy			0.81	7365	
macro avg	0.74	0.80	0.76	7365	
weighted avg	0.85	0.81	0.82	7365	

Classification Report for RandomForestClassifier					
	precision	recall	f1-score	support	
0	0.93	0.98	0.95	5749	
1	0.90	0.74	0.81	1616	
accuracy			0.92	7365	
macro avg	0.91	0.86	0.88	7365	
weighted avg	0.92	0.92	0.92	7365	

We observe that random forest performs better than logistic regression on unseen data. You are tasked to plot the receiver operation curve and the confusion matrix of the two models.

In the next step we will train deep learning models to estimate probability of default.

Deep Learning

We start by importing `keras` classifier to run the model.

```
In [25]: # !pip install tensorflow
# !pip install scikeras
```

```
In [26]: import logging

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Dropout
# from tensorflow.keras.wrappers.scikit_learn import KerasClassifier
from scikeras.wrappers import KerasClassifier, KerasRegressor

tf.get_logger().setLevel(logging.ERROR)
```

```

2025-10-15 13:16:05.395032: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
2025-10-15 13:16:05.395533: I tensorflow/core/util/port.cc:153] oneDNN custom ope
rations are on. You may see slightly different numerical results due to floating-
point round-off errors from different computation orders. To turn them off, set t
he environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-15 13:16:05.444465: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in performa
nce-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512
_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild Tens
orFlow with the appropriate compiler flags.
2025-10-15 13:16:06.683833: I tensorflow/core/util/port.cc:153] oneDNN custom ope
rations are on. You may see slightly different numerical results due to floating-
point round-off errors from different computation orders. To turn them off, set t
he environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-15 13:16:06.684609: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.

```

We then write a function with the model layers and activation functions at each layer

```

In [27]: def model_function(dropout_rate, verbose=0):
# Select a `keras` model, sequential function allows us to specify our neural
model = keras.Sequential()
# First Layer is dense implying that the features are connected to every sin
model.add(Dense(128, kernel_initializer="normal", activation="relu", input_dim=100))
model.add(Dense(64, kernel_initializer="normal", activation="relu"))
model.add(Dense(8, kernel_initializer="normal", activation="relu"))
# Drop out is added to ignore irrelevant neurons
model.add(Dropout(dropout_rate))
# We add sigmoid to classify the target
model.add(Dense(1, activation="sigmoid"))
# the loss function is binary cross entropy because the problem is a binary
model.compile(loss="binary_crossentropy", optimizer="rmsprop")
return model

```

The next cell defines the model parameters and hyperparameters, train the model and validate the model on test dataset.

```

In [28]: model = KerasClassifier(
    build_fn=model_function, dropout_rate=0.2, verbose=0, batch_size=50, epochs=100
)
model.fit(X_train, y_train)
y_predict_dl = model.predict(X_test)

```

```

/usr/local/lib/python3.11/site-packages/scikeras/wrappers.py:925: UserWarning: ``
build_fn`` will be renamed to ``model`` in a future release, at which point use o
f ``build_fn`` will raise an Error instead.
  X, y = self._initialize(X, y)
/usr/local/lib/python3.11/site-packages/keras/src/layers/core/dense.py:92: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using S
equential models, prefer using an `Input(shape)` object as the first layer in the
model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-10-15 13:16:06.873891: E external/local_xla/xla/stream_executor/cuda/cuda_pl
atform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)

```

We now evaluate the model using several metrics

```
In [29]: ROC_AUC_metric = roc_auc_score(y_test, pd.DataFrame(y_predict_dl.flatten()))
print("Deep Learning ROC AUC is {:.4f}".format(ROC_AUC_metric))
```

Deep Learning ROC AUC is 0.8366

```
In [30]: print("\nClassification Report for deep learning model")
print(classification_report(y_test, y_predict_dl.flatten()))
```

Classification Report for deep learning model

	precision	recall	f1-score	support
0	0.92	0.98	0.95	5749
1	0.92	0.69	0.79	1616
accuracy			0.92	7365
macro avg	0.92	0.84	0.87	7365
weighted avg	0.92	0.92	0.91	7365

```
In [31]: print("\nThe brier score is equal to ")
brier_score_loss(y_test, y_predict_dl.flatten(), pos_label=1)
```

The brier score is equal to

Out[31]: 0.08119484046164291

We can observe that the model performance is comparable to random forest model without much data preparation or data balancing.

The next step is to help a business make the right decision. The discussion for this is beyond the scope of this lesson and we will therefore only give a brief discussion of what to expect.

2.7 Business Decision Making

The final step of modeling is to evaluate if our modeling met the set Key Performance Indicators (KPIs) by the institution. Among the metrics to measure are:

2.7.1 Acceptance Rate

The threshold for an acceptance rate is used for setting the percentage of new loans business are willing to disburse. We use test data as a proxy for fresh new loans to measure the performance.

We use the threshold to assign new `loan_status` values. We then observe if the distribution of the bads and goods changes.

2.7.2 Bad Rates

Now that we have the acceptance rate, we do the analysis on bad rate percentages within the loans that met the threshold. This allows us to see the percentage of defaults that our model will accept.

In summary, we set an acceptance rate so as to have fewer bad loans in our portfolio.

2.7.3 Strategy

At this point, a strategy meeting with project (business) owners is called where we brainstorm with them and strategize on the best approach for using the scorecard. This is done by building a strategy table that has simulations with possible outcomes.

2.7.4 Total Loss

Finally, we estimate the total expected loss having concrete decisions from the data science team and input from business owners. We define the expected loss as the amount of money a financial lender might lose by lending to a borrower. The components of expected loss (EL) are:

- Probabilities of default (**PD**)
- The loss given default (**LGD**)
- The `loan_amnt`, which will be assumed to be the exposure at default (**EAD**).

The factors that contribute to expected loss are categorized into:

1. Borrower specific factors.
2. Economic environment.

The expected loss is calculated as shown below:

$$\text{Total Expected Loss} = \sum_{x=1}^n PD_x * LGD_x * EAD_x$$

$$\text{expected_loss} = \text{prob_default} * \text{lgd} * \text{loan_amnt}$$

Probability of default as defined earlier is the inability of the borrower to repay their debt in full or on time/ schedule.

Loss given default on the other hand is the proportion of the borrowed amount by the customer that the financial institution will be unable to recover once the customer defaults.

Exposure at default is the total sum that a lender is exposed to at the time of default.

Let us consider an example below.

Consider the case of a mortgage worth \$1,000,000 and the lender is willing to finance 80% of the mortgage value. The borrower pays back \$100,000 and our model shows that the probability of default is 20%. Further assume that the bank can recover some of the money by auctioning the house at \$560,000 after the customer has defaulted. Then, this is how we will calculate the expected loss:

- Cost of House = \$1,000,000
- Lender finances 80% of the cost, that is, loan amount = \$800,000
- If the borrower pays back \$100,000 then exposure at default = \$800,000 - \$100,000 = \$700,000
- Probability of default = 0.2

- Loss given default = $\frac{\$700,000 - \$560,000}{\$700,000} = 20\%$

So the expected loss is: $\text{PD} \times \text{LGD} \times \text{EAD} = 0.2 \times 0.2 \times \$700,000 = \$28,000$

In this lesson, we have covered an application of machine learning in credit risk management. Can you think of how the same can be implemented using a deep learning algorithm?

3. Conclusion

This module introduced the concept of deep learning and its application in finance. In the next module, we will study hyperparameter tuning in machine learning models to improve their performance.

References

1. Cooper, Michael James. *A Deep Learning Prediction Model for Mortgage Default*. 2018. University of Bristol, Masters thesis.
2. Giesecke, Kay. "Credit Risk Modeling and Valuation: An Introduction." 2004. Available at SSRN 479323.
3. Sirignano, Justin et al. "Deep Learning for Mortgage Risk." *arXiv preprint arXiv:1607.02470*. 2016.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.